International Islamic University Chittagong
Department of Computer Science & Engineering
Autumn - 2024

Course Code: CSE-2321
Course Title: Data Structures

Mohammed Shamsul Alam
Professor, Dept. of CSE, IIUC

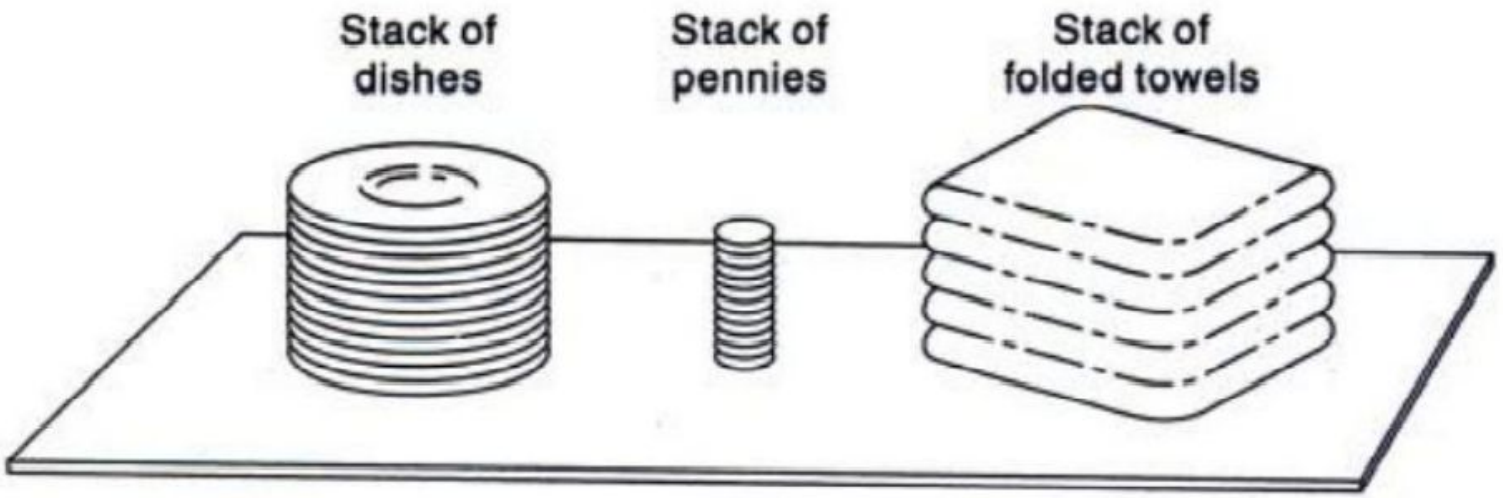
Lecture – 9

Stacks

Stacks

A **stack** is a list of elements in which an element may be inserted or deleted only at one end, called the ***top*** of the stack.

Stacks are also called ***last-in first-out*** **(LIFO)** lists.



Example 6.1

Suppose the following 6 elements are pushed, in order, onto an empty stack:

AAA, BBB, CCC, DDD, EEE, FFF

Figure 6.3 shows three ways of picturing such a stack. For notational convenience, we will frequently designate the stack by writing:

STACK: AAA, BBB, CCC, DDD, EEE, FFF

Stacks

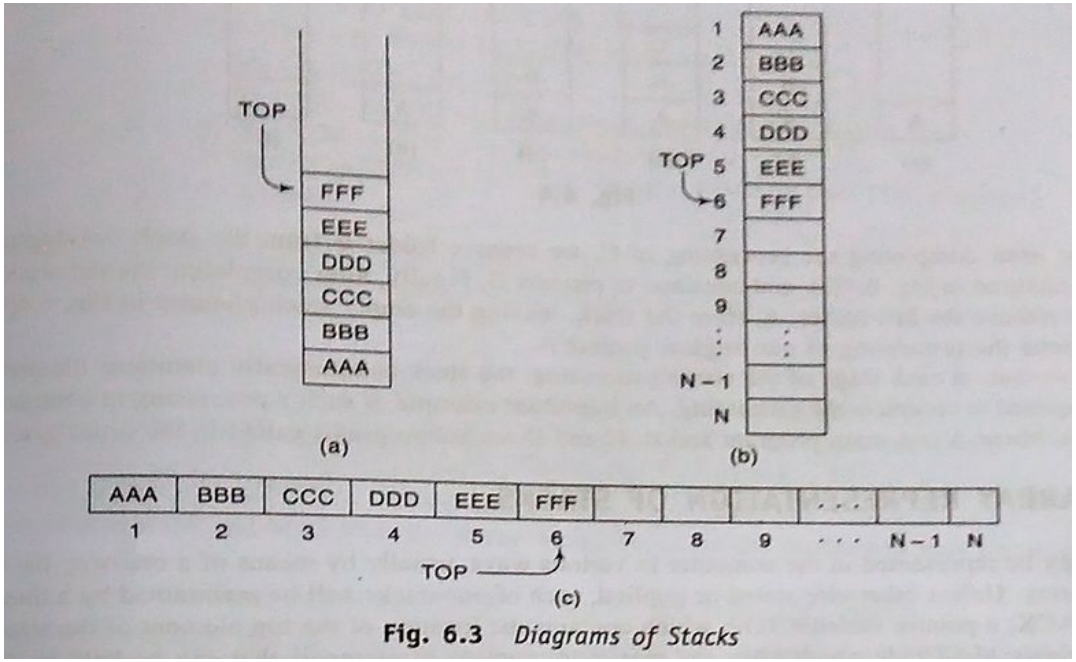


Fig. 6.3 *Diagrams of Stacks*

- There are two basic operations are associated with stacks:
  - PUSH - to insert an element into a stack
  - POP - to delete an element from a stack

- Stacks may be represented in the computer in various ways, usually by means of

  i) a Linear Array or

  ii) a one-way Linked List

Array Representation of Stacks

Stack can be represented by a linear array **STACK**, a pointer variable **TOP**, which contains the location of the top elements of the stack; and a variable **MAXSTK** which gives the maximum number of elements that can be held by the stack. The condition, TOP = 0 will indicate that the stack is empty.

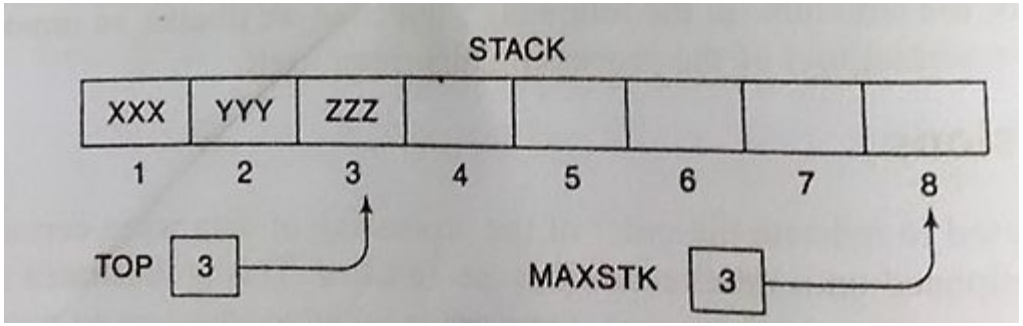


Fig shows such an array representation of a stack. Since TOP = 3, the attacks has three elements XXX, YYY and ZZZ. And since MAXSTK = 8, we can add 5 more items in the stack.

Array Representation of Stacks

Algorithm 6.1: PUSH (STACK, TOP, MAXSTK, ITEM)

This procedure pushes an item on to a stack.

1. [Stack already filled]? If TOP = MAXSTK, then: Print: OVERFLOW, and Return.
2. Set TOP: = TOP + 1. [Increase TOP by 1].
3. Set STACK [TOP]: = ITEM. [Inserts ITEM in new TOP position].
4. Return

Algorithm 6.2: POP (STACK, TOP, ITEM)

This procedure deletes the TOP element of STACK and assigns it to the variable ITEM.

- [Stack has an item to be removed] If TOP = 0, then: Print: UNDERFLOW and Return.

1. Set ITEM: =STACK [TOP]. [Assign TOP element to ITEM].
2. Set TOP: = TOP – 1 [Decrease TOP by 1].
3. Return.

Array Representation of Stacks

6.1 Consider the following stack of characters, where STACK is allocated N = 8 memory cells:

STACK: A, C, D, F, K, ___, ___, ___,

(For notational convenience, we use "___" to denote an empty memory cell.) Describe the stack as the following operations take place:

(a) POP(STACK, ITEM)
(b) POP(STACK, ITEM)
(c) PUSH(STACK, L)
(d) PUSH(STACK, P)
(e) POP(STACK, ITEM)
(f) PUSH(STACK, R)
(g) PUSH(STACK, S)
(h) POP(STACK, ITEM)

The POP procedure always deletes the top element from the stack, and the PUSH procedure always adds the new element to the top of the stack. Accordingly:

(a) STACK: A, C, D, F, ___, ___, ___, ___
(b) STACK: A, C, D, ___, ___, ___, ___, ___
(c) STACK: A, C, D, L, ___, ___, ___, ___
(d) STACK: A, C, D, L, P, ___, ___, ___
(e) STACK: A, C, D, L, ___, ___, ___, ___
(f) STACK: A, C, D, L, R, ___, ___, ___
(g) STACK: A, C, D, L, R, S, ___, ___
(h) STACK: A, C, D, L, R, ___, ___, ___

Minimizing Overflow

Minimizing Overflow

There is an essential difference between underflow and overflow in dealing with stacks. Underflow depends exclusively upon the given algorithm and the given input data, and hence there is no direct control by the programmer. Overflow, on the other hand, depends upon the arbitrary choice of the programmer for the amount of memory space reserved for each stack, and this choice does influence the number of times overflow may occur.

Generally speaking, the number of elements in a stack fluctuates as elements are added to or removed from a stack. Accordingly, the particular choice of the amount of memory for a given stack involves a time-space tradeoff. Specifically, initially reserving a great deal of space for each stack will decrease the number of times overflow may occur; however, this may be an expensive use of the space if most of the space is seldom used. On the other hand, reserving a small amount of space for each stack may increase the number of times overflow occurs; and the time required for resolving an overflow, such as by adding space to the stack, may be more expensive than the space saved.

Various techniques have been developed which modify the array representation of stacks so that the amount of space reserved for more than one stack may be more efficiently used. Most of these techniques lie beyond the scope of this text. We do illustrate one such technique in the following example.

Example 6.3

Suppose a given algorithm requires two stacks, A and B. One can define an array STACKA with n_1 elements for stack A and an array STACKB with n_2 elements for stack B. Overflow will occur when either stack A contains more than n_1 elements or stack B contains more than n_2 elements.

Suppose instead that we define a single array STACK with $n = n_1 + n_2$ elements for stacks A and B together. As pictured in Fig. 6.6, we define STACK[1] as the bottom of stack A and let A "grow" to the right, and we define STACK[n] as the bottom of stack B and let B "grow" to the left. In this case, overflow will occur only when A and B together have more than $n = n_1 + n_2$ elements. This technique will usually decrease the number of times overflow occurs even though we have not increased the total amount of space reserved for the two stacks. In using this data structure, the operations of PUSH and POP will need to be modified.

Polish Notation

Example 6.5

Suppose we want to evaluate the following parenthesis-free arithmetic expression:

$$2 \uparrow 3 + 5 * 2 \uparrow 2 - 12 / 6$$

First we evaluate the exponentiations to obtain

$$8 + 5 * 4 - 12 / 6$$

Then we evaluate the multiplication and division to obtain $8 + 20 - 2$. Last, we evaluate the addition and subtraction to obtain the final result, 26. Observe that the expression is traversed three times, each time corresponding to a level of precedence of the operations.

Polish Notation

For most common arithmetic operations, the operator symbol is placed between its two operands. For example,

$$A + B \qquad C - D \qquad E * F \qquad G/H$$

This is called *infix notation*. With this notation, we must distinguish between

$$(A + B) * C \qquad \text{and} \qquad A + (B * C)$$

by using either parentheses or some operator-precedence convention such as the usual precedence levels discussed above. Accordingly, the order of the operators and operands in an arithmetic expression does not uniquely determine the order in which the operations are to be performed.

Polish notation, named after the Polish mathematician Jan Lukasiewicz, refers to the notation in which the operator symbol is placed before its two operands. For example,

$$+AB \qquad -CD \qquad *EF \qquad /GH$$

We translate, step by step, the following infix expressions into Polish notation using brackets [ ] to indicate a partial translation:

$$(A + B) * C = [+AB] * C = *+ ABC$$
$$A + (B * C) = A + [*BC) = + A*BC$$
$$(A + B)/(C - D) = [+AB]/[-CD] = / + AB - CD$$

The fundamental property of Polish notation is that the order in which the operations are to be performed is completely determined by the positions of the operators and operands in the expression. Accordingly, one never needs parentheses when writing expressions in Polish notation.

Reverse Polish notation refers to the analogous notation in which the operator symbol is placed after its two operands:

$$AB+ \qquad CD- \qquad EF* \qquad GH/$$

Again, one never needs parentheses to determine the order of the operations in any arithmetic expression written in reverse Polish notation. This notation is frequently called *postfix* (or *suffix*) *notation*, whereas *prefix notation* is the term used for Polish notation, discussed in the preceding paragraph.

Evaluation of a Postfix notation

Evaluation of a Postfix Expression

Suppose P is an arithmetic expression written in postfix notation. The following algorithm, which uses a STACK to hold operands, evaluates P.

Algorithm 6.5: This algorithm finds the VALUE of an arithmetic expression P written in postfix notation.

1. Add a right parenthesis ")" at the end of P. [This acts as a sentinel.]
2. Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel ")" is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator $\otimes$ is encountered, then:
   (a) Remove the two top elements of STACK, where A is the top element and B is the next-to-top element.
   (b) Evaluate $B \otimes A$.
   (c) Place the result of (b) back on STACK.
   [End of If structure.]

   [End of Step 2 loop.]
5. Set VALUE equal to the top element on STACK.
6. Exit.

Evaluation of a Postfix notation

Example 6.6

Consider the following arithmetic expression P written in postfix notation:

P: 5, 6, 2, +, *, 12, 4, /, −

(Commas are used to separate the elements of P so that 5, 6, 2 is not interpreted as the number 562.) The equivalent infix expression Q follows:

Q: 5 * (6 + 2) − 12 / 4

Note that parentheses are necessary for the infix expression Q but not for the postfix expression P.

We evaluate P by simulating Algorithm 6.5. First we add a sentinel right parenthesis at the end of P to obtain

P:	5,	6,	2,	+,	*,	12,	4,	/,	−,	)
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)

The elements of P have been labeled from left to right for easy reference. Figure 6.11 shows the contents of STACK as each element of P is scanned. The final number in STACK, 37, which is assigned to VALUE when the sentinel ")" is scanned, is the value of P.

Symbol Scanned		STACK
(1)	5	5
(2)	6	5, 6
(3)	2	5, 6, 2
(4)	+	5, 8
(5)	*	40
(6)	12	40, 12
(7)	4	40, 12, 4
(8)	/	40, 3
(9)	−	37
(10)	)	

Transforming Infix Expressions into Postfix Expressions

Algorithm 6.6: POLISH(Q, P)

Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

1. Push "(" onto STACK, and add ")" to the end of Q.
2. Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:
3. If an operand is encountered, add it to P.
4. If a left parenthesis is encountered, push it onto STACK.
5. If an operator ⊗ is encountered, then:
   - (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than ⊗.
   - (b) Add ⊗ to STACK.

   [End of If structure.]
6. If a right parenthesis is encountered, then:
   - (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
   - (b) Remove the left parenthesis. [Do not add the left parenthesis to P.]

   [End of If structure.]

   [End of Step 2 loop.]
7. Exit.

Transforming Infix Expressions into Postfix Expressions

Example 6.7

Consider the following arithmetic infix expression Q:

Q: A + (B * C – (D / E ↑ F) * G) * H

We simulate Algorithm 6.6 to transform Q into its equivalent postfix expression P. First we push "(" onto STACK, and then we add ")" to the end of Q to obtain:

Q:	A	+	(	B	*	C	–	(	D	/	E	↑	F	)	*
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)	(15)
											G	)	*	H	)
											(16)	(17)	(18)	(19)	(20)

Symbol Scanned		STACK	Expression P
(1)	A	(	A
(2)	+	(+	A
(3)	(	(+ (	A
(4)	B	(+ (	A B
(5)	*	(+ (*	A B
(6)	C	(+ (*	A B C
(7)	–	(+ (–	A B C *
(8)	(	(+ (– (	A B C *
(9)	D	(+ (– (	A B C * D
(10)	/	(+ (– (/	A B C * D
(11)	E	(+ (– (/	A B C * D E
(12)	↑	(+ (– (/ ↑	A B C * D E
(13)	F	(+ (– (/ ↑	A B C * D E F
(14)	)	(+ (–	A B C * D E F ↑ /
(15)	*	(+ (– *	A B C * D E F ↑ /
(16)	G	(+ (– *	A B C * D E F ↑ / G
(17)	)	(+	A B C * D E F ↑ / G * –
(18)	*	(+ *	A B C * D E F ↑ / G * –
(19)	H	(+ *	A B C * D E F ↑ / G * – H
(20)	)		A B C * D E F ↑ / G * – H * +

Lecture – 10

Recursion

Recursion

- A function that **calls itself** is termed as a ***recursive function*** and the process involved in doing this is called **recursion**.
- Recursion is an essential concept in Computer Science.
- Recursion makes it convenient to solve various problems that would be cumbersome to solve using iterative constructs such as for, while, and do while.
- Recursion is a methodology that permits breaking down of a problem into one or many sub problems that are similar to the original problem

a function is said to be *recursively defined* if the function definition refers to itself. Again, in order for the definition not to be circular, it must have the following two properties:

(1) There must be certain arguments, called *base values*, for which the function does not refer to itself.

(2) Each time the function does refer to itself, the argument of the function must be closer to a base value

A recursive function with these two properties is also said to be well-defined.

Factorial Function

- A classic example of recursion is the definition of the **factorial** function.

Definition 6.1: (Factorial Function)

(a) If $n = 0$, then $n! = 1$.
(b) If $n > 0$, then $n! = n \cdot (n - 1)!$

Observe that this definition of $n!$ is recursive, since it refers to itself when it uses $(n - 1)!$ However, (a) the value of $n!$ is explicitly given when $n = 0$ (thus 0 is the base value); and (b) the value of $n!$ for arbitrary n is defined in terms of a smaller value of n which is closer to the base value 0. Accordingly, the definition is not circular, or in other words, the procedure is well-defined.

(1) $4! = 4 \cdot 3!$
(2) $3! = 3 \cdot 2!$
(3) $2! = 2 \cdot 1!$
(4) $1! = 1 \cdot 0!$
(5) $0! = 1$
(6) $1! = 1 \cdot 1 = 1$
(7) $2! = 2 \cdot 1 = 2$
(8) $3! = 3 \cdot 2 = 6$
(9) $4! = 4 \cdot 6 = 24$

Factorial Function

The following are two procedures that each calculate *n* factorial.

Procedure 6.9A: FACTORIAL(FACT, N)

This procedure calculates N! and returns the value in the variable FACT.

1. If N = 0, then: Set FACT := 1, and Return.
2. Set FACT := 1 [Initializes FACT for loop.]
3. Repeat for K = 1 to N.
   Set FACT := K*FACT.
   [End of loop.]
4. Return.

Procedure 6.9B: FACTORIAL(FACT, N)

This procedure calculates N! and returns the value in the variable FACT.

1. If N = 0, then: Set FACT := 1, and Return.
2. Call FACTORIAL(FACT, N – 1).
3. Set FACT := N*FACT.
4. Return.

Fibonacci Sequence

The celebrated Fibonacci sequence (usually denoted by $F_0, F_1, F_2, \ldots$) is as follows:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots.$$

That is, $F_0 = 0$ and $F_1 = 1$ and each succeeding term is the sum of the two preceding terms. For example, the next two terms of the sequence are

$$34 + 55 = 89 \quad \text{and} \quad 55 + 89 = 144$$

A formal definition of this function follows:

Definition 6.2: (Fibonacci Sequence)

(a) If $n = 0$ or $n = 1$, then $F_n = n$.
(b) If $n > 1$, then $F_n = F_{n-2} + F_{n-1}$.

This is another example of a recursive definition, since the definition refers to itself when it uses F_{n-2} and F_{n-1}. Here (a) the base values are 0 and 1, and (b) the value of F_n is defined in terms of smaller values of n which are closer to the base values. Accordingly, this function is well-defined.

Fibonacci Sequence

FIBONACCI(F, N)
This procedure finds the first N Fibonacci numbers and assigns them to an array F.

1. Set F[1] := 1 and F[2] := 1.
2. Repeat for L = 3 to N:
   Set F[L] := F[L – 1] + F[L – 2].
   [End of loop.]
3. Return.

A procedure for finding the nth term F_n of the Fibonacci sequence follows.

Procedure 6.10: FIBONACCI(FIB, N)
This procedure calculates F_N and returns the value in the first parameter FIB.

1. If N = 0 or N = 1, then: Set FIB := N, and Return.
2. Call FIBONACCI(FIBA, N – 2).
3. Call FIBONACCI(FIBB, N – 1).
4. Set FIB := FIBA + FIBB.
5. Return.

This is another example of a recursive procedure, since the procedure contains a call to itself.

Divide & Conquer Algorithms

Consider a problem P associated with a set S. Suppose A is an algorithm which partitions S into smaller sets such that the solution of the problem P for S is reduced to the solution of P for one or more of the smaller sets. Then A is called a divide-and-conquer algorithm.

Ackermann Function

The Ackermann function is a function with two arguments each of which can be assigned any nonnegative integer: 0, 1, 2,.... This function is defined as follows:

Definition 6.3: (Ackermann Function)

(a) If $m = 0$, then $A(m, n) = n + 1$.
(b) If $m \neq 0$ but $n = 0$, then $A(m, n) = A(m - 1, 1)$.
(c) If $m \neq 0$ and $n \neq 0$, then $A(m, n) = A(m - 1, A(m, n - 1))$

Find the value of A(1,3).

Ackermann Function

Find the value of A(1,3).

(1) $A(1, 3) = A(0, A(1, 2))$
(2) $A(1, 2) = A(0, A(1, 1))$
(3) $A(1, 1) = A(0, A(1, 0))$
(4) $A(1, 0) = A(0, 1)$
(5) $A(0, 1) = 1 + 1 = 2$
(6) $A(1, 0) = 2$
(7) $A(1, 1) = A(0, 2)$
(8) $A(0, 2) = 2 + 1 = 3$
(9) $A(1, 1) = 3$
(10) $A(1, 2) = A(0, 3)$
(11) $A(0, 3) = 3 + 1 = 4$
(12) $A(1, 2) = 4$
(13) $A(1, 3) = A(0, 4)$
(14) $A(0, 4) = 4 + 1 = 5$
(15) $A(1, 3) = 5$

Towers of Hanoi

Suppose three pegs, labeled A, B and C, are given, and suppose on peg A there are placed a finite number n of disks with decreasing size. This is pictured in Fig. 6.14 for the case $n = 6$. The object of the game is to move the disks from peg A to peg C using peg B as an auxiliary. The rules of the game are as follows:

(a) Only one disk may be moved at a time. Specifically, only the top disk on any peg may be moved to any other peg.

(b) At no time can a larger disk be placed on a smaller disk.

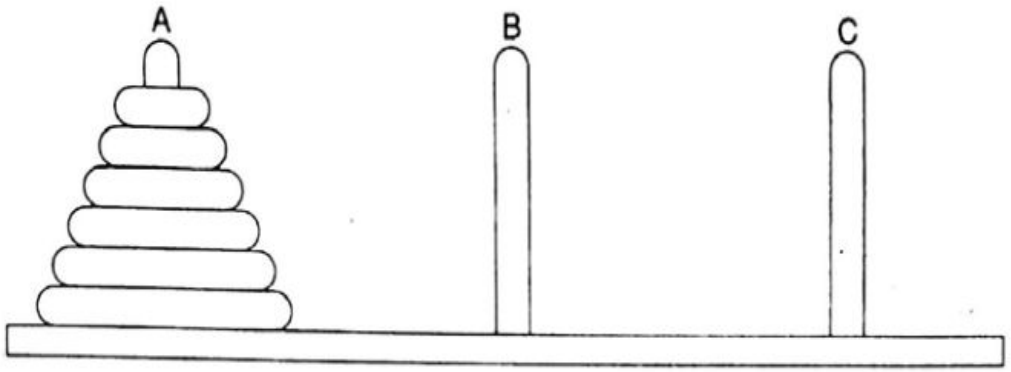


Fig. 6.14 *Initial Setup of Towers of Hanoi with n = 6*

Towers of Hanoi

TOWER(N, BEG, AUX, END)
This procedure gives a recursive solution to the Towers of Hanoi problem for N disks.

1. If N = 1, then:
   (a) Write: BEG → END.
   (b) Return.
   [End of If structure.]
2. [Move N – 1 disks from peg BEG to peg AUX.]
   Call TOWER(N – 1, BEG, END, AUX).
3. Write: BEG → END.
4. [Move N – 1 disks from peg AUX to peg END.]
   Call TOWER(N – 1, AUX, BEG, END).
5. Return.

Towers of Hanoi

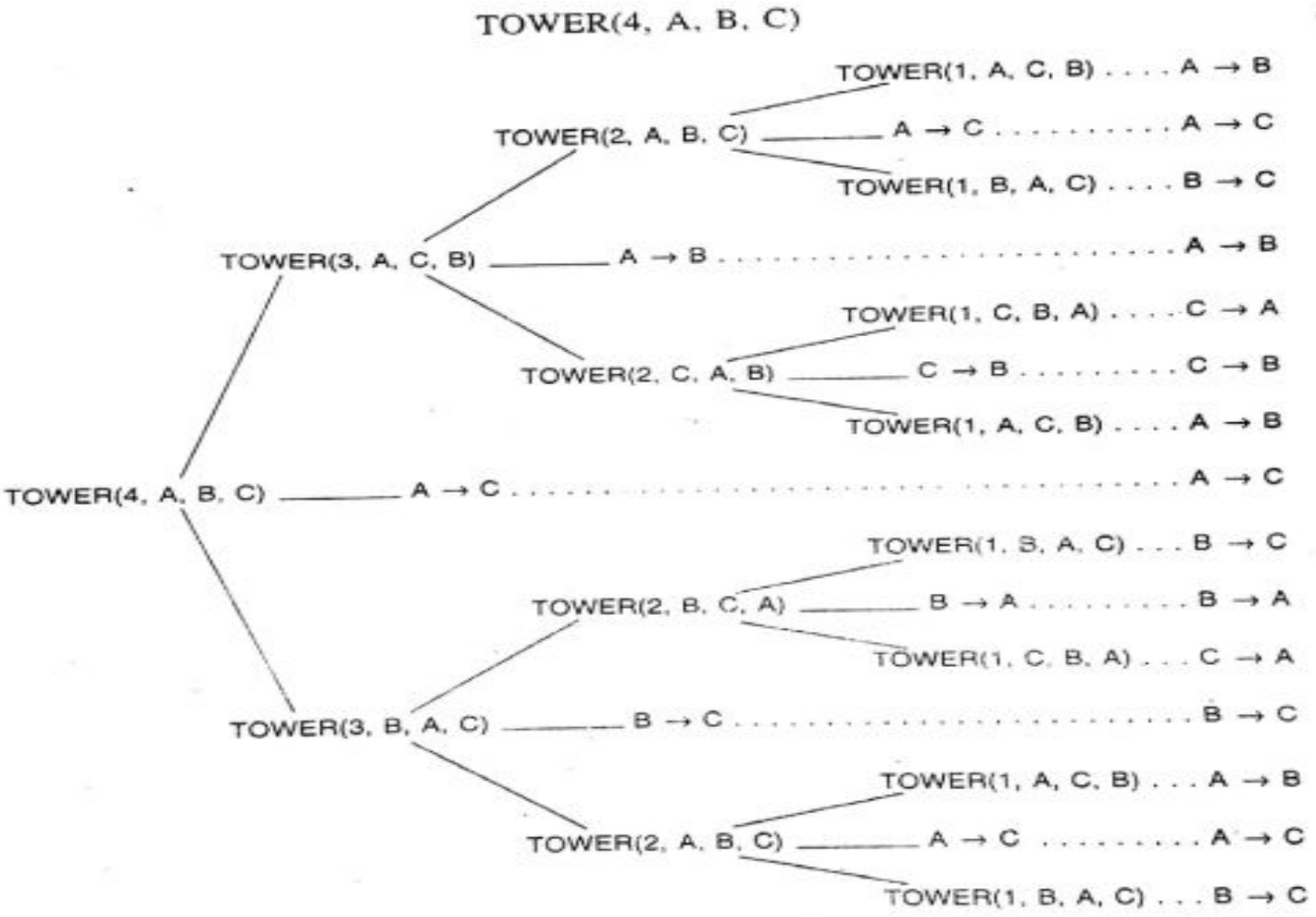


Fig. 6.17 *Recursive Solution to Towers of Hanoi Problem for n = 4*

Observe that the recursive solution for $n = 4$ disks consists of the following 15 moves:

$A \to B \quad A \to C \quad B \to C \quad A \to B \quad C \to A \quad C \to B \quad A \to B \quad A \to C$
$B \to C \quad B \to A \quad C \to A \quad B \to C \quad A \to B \quad A \to C \quad B \to C$

In general, this recursive solution requires $f(n) = 2^n - 1$ moves for n disks.

Lecture – 11

Queues

Queues

A **Queue** is a linear data structure in which the insertion and deletion operations are performed at two different ends.

- In a queue data structure, the insertion operation is performed at a position which is known as **'rear'** and the deletion operation is performed at a position which is known as **'front'**.
- Queue **follows FIFO (First In First Out)** principle.

- The elements in a queue are processed like customers standing in a grocery check-out line: the first customer in line is the first one served.

Applications of Queues

There are numerous applications of queue in computer science.

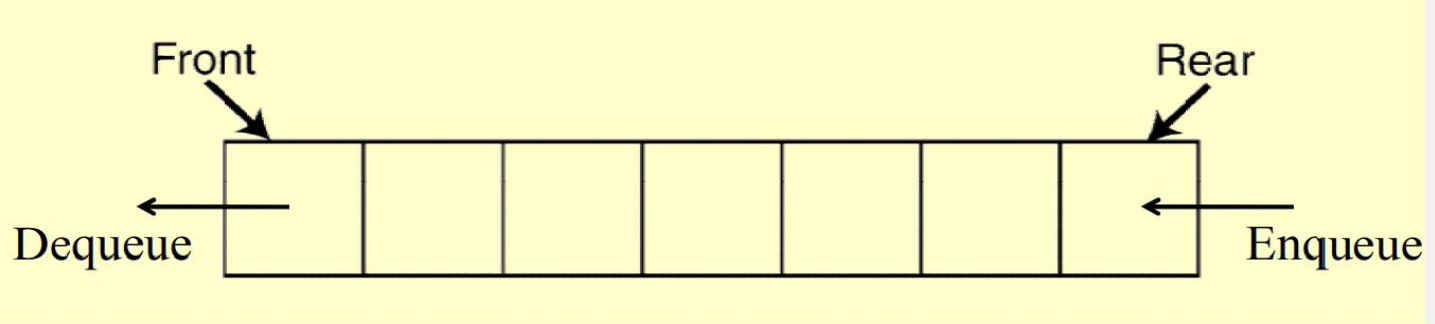
- Various real-life applications, like railway ticket reservation, banking system are implemented using queue.
- One of the most useful applications of queue is in simulation.
- Another application of queue is in operating system to implement various functions like CPU scheduling in multiprogramming environment, device management (printer or disk), etc.
- Communications software also uses queues to hold information received over networks and dialup connections. Sometimes information is transmitted to a system faster than it can be processed, so it is placed in a queue when it is received.
- Besides these, there are several algorithms like level-order traversal of binary tree, breadh-first search in graphs, etc., that use queues to solve the problems efficiently.

Representation of Queues

Queues may be represented in the computer in various ways, usually by means at one-way lists or linear arrays. Unless otherwise stated or implied, each of our queues will be maintained by a linear array QUEUE and two pointer variables: FRONT, containing the location of the front element of the queue; and REAR, containing .the location of the rear element of the queue. The condition FRONT = NULL will indicate that the queue is empty.

Queue Operations

The two main operations of queue are insertion and deletion of items which are referred as **enqueue** and **dequeue** respectively. In *enqueue* operation, an item is **added** to the **rear** end of the queue. In *dequeue* operation, the item is **deleted** from the **front** end of the queue.



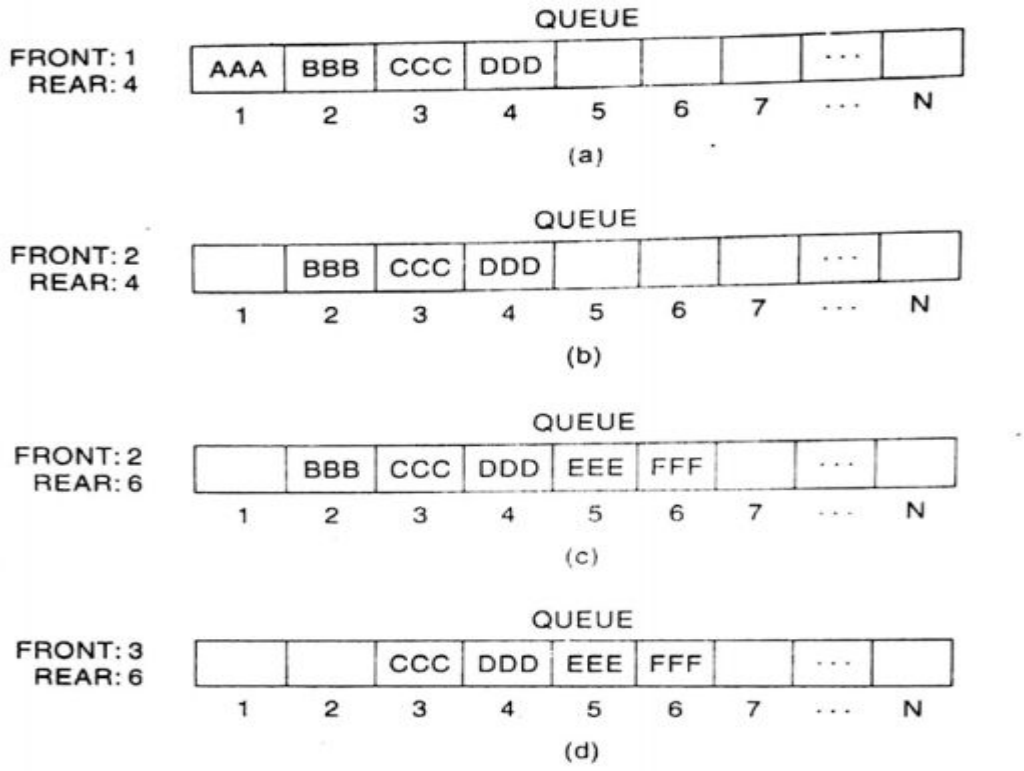
Insert at Rear End

To insert an item into the queue, first it should be verified whether the queue is full or not. If the queue is full, a new item cannot be inserted into the queue. The condition ***FRONT=NULL indicates that the queue is empty***. If the queue is not full, items are inserted at the rear end. When an item is added to the queue, the value of rear is incremented by 1 i.e. **REAR := REAR +1**

Queue Operations

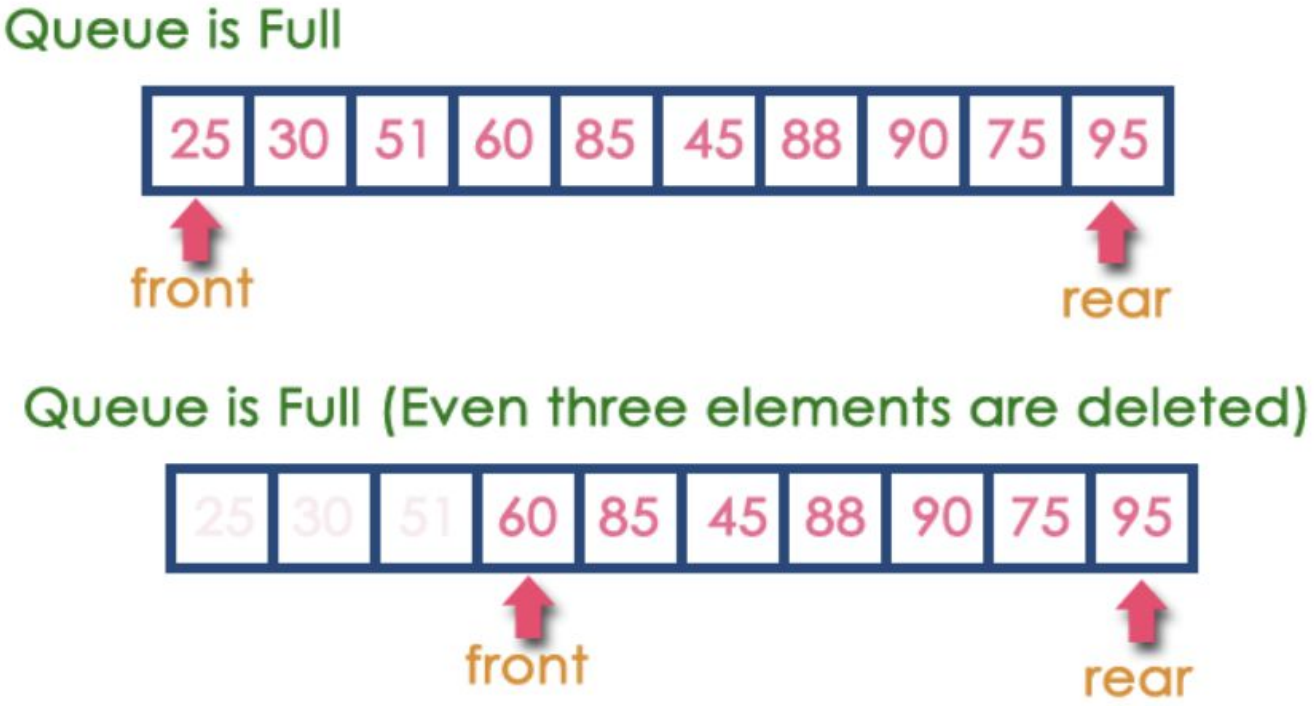
Delete from the Front End

To delete an item from the stack, first it should be verified that the queue is not empty. If the queue is not empty, the items are deleted at the front end of the queue. When an item is deleted from the queue, the value of the front is incremented by 1 i.e **FRONT := FRONT + 1**



Problem with Normal Queue

In a normal Queue Data Structure, we can insert elements until queue becomes full. But once if queue becomes full, we can not insert the next element until all the elements are deleted from the queue

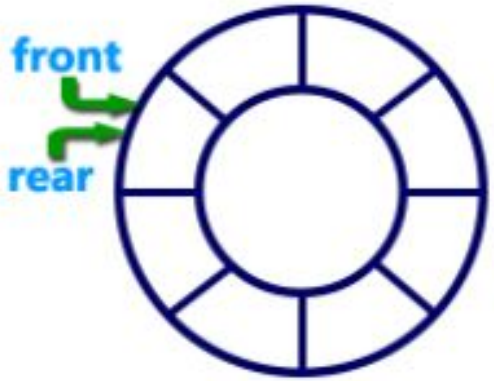


'rear' is still at last position. In above situation, *even though we have empty positions in the queue we can not make use of them to insert new element*.

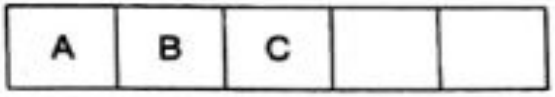
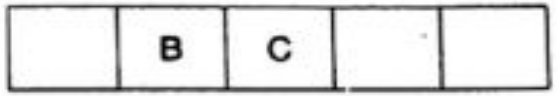
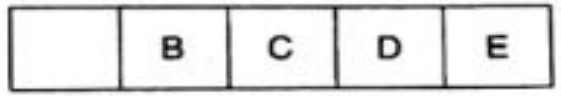
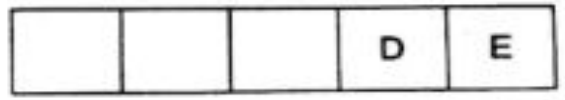
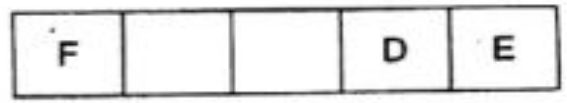
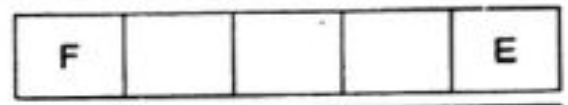
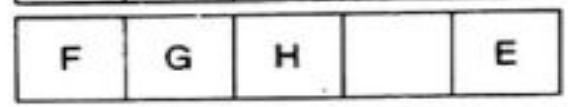
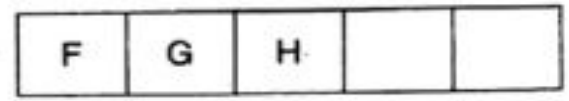
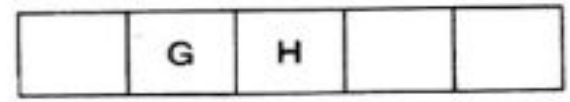
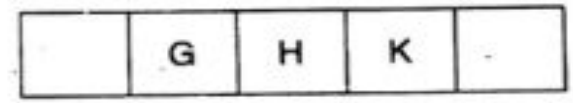
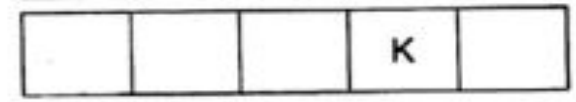
Circular Queue

A **Circular Queue** is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle, that is, **QUEUE[1] comes after QUEUE[N].**

Graphical representation of a circular queue is as follows:



- If REAR = N and an ITEM is inserted into the QUEUE when there is free space in the QUEUE, then we reset REAR = 1 instead of increasing REAR to N+1.
- Similarly, if FRONT = N and an element of QUEUE is deleted, we reset FRONT = 1 instead of increasing FRONT to N+1.

		QUEUE
(a) Initially empty:	FRONT: 0 REAR: 0	
		1 2 3 4 5
(b) A, B and then C inserted:	FRONT: 1 REAR: 3	
(c) A deleted:	FRONT: 2 REAR: 3	
(d) D and then E inserted:	FRONT: 2 REAR: 5	
(e) B and C deleted:	FRONT: 4 REAR: 5	
(f) F Inserted:	FRONT: 4 REAR: 1	
(g) D deleted:	FRONT: 5 REAR: 1	
(h) G and then H inserted:	FRONT: 5 REAR: 3	
(i) E deleted:	FRONT: 1 REAR: 3	
(j) F deleted:	FRONT: 2 REAR: 3	
(k) K inserted:	FRONT: 2 REAR: 4	
(l) G and H deleted:	FRONT: 4 REAR: 4	
(m) K deleted, QUEUE empty:	FRONT: 0 REAR: 0	

Queue Operations

QINSERT(QUEUE, N, FRONT, REAR, ITEM)
This procedure inserts an element ITEM into a queue.

1. [Queue already filled?]
   If FRONT = 1 and REAR = N, or if FRONT = REAR + 1, then:
   Write: OVERFLOW, and Return.

2. [Find new value of REAR.]
   If FRONT := NULL, then: [Queue initially empty.]
   Set FRONT := 1 and REAR := 1.
   Else if REAR = N, then:
   Set REAR := 1.
   Else:
   Set REAR := REAR + 1.
   [End of If structure.]
3. Set QUEUE[REAR] := ITEM. [This inserts new element.]
4. Return.

: QDELETE(QUEUE, N, FRONT, REAR, ITEM)
This procedure deletes an element from a queue and assigns it to the variable ITEM.

1. [Queue already empty?]
   If FRONT := NULL, then: Write: UNDERFLOW, and Return.
2. Set ITEM := QUEUE[FRONT].
3. [Find new value of FRONT.]
   If FRONT = REAR, then: [Queue has only one element to start.]
   Set FRONT := NULL and REAR := NULL.
   Else if FRONT = N, then:
   Set FRONT := 1.
   Else:
   Set FRONT := FRONT + 1.
   [End of If structure.]
4. Return.

Linked Representation of Queues

In this section we discuss the linked representation of a queue. A linked queue is a queue implemented as a linked list with two pointer variables FRONT and REAR pointing to the nodes which is in the FRONT and REAR of the queue. The INFO fields of the list hold the elements of the queue and the LINK fields hold pointers to the neighboring elements in the queue. Fig. 6.22 illustrates the linked representation of the queue shown in Fig. 6.16(a).

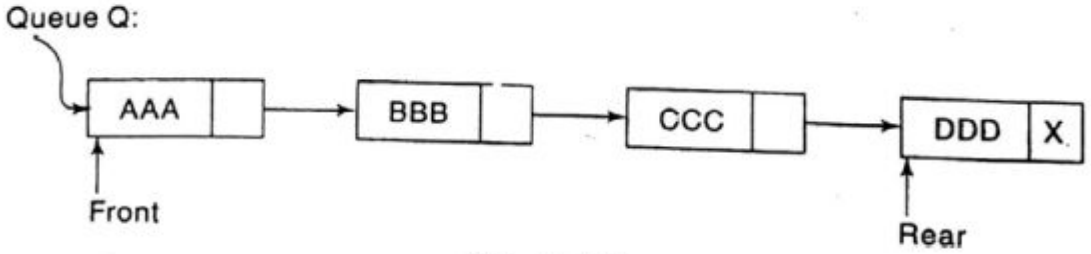


Fig. 6.22

Procedure 6.15: LINKQ_INSERT(INFO,LINK, FRONT, REAR,AVAIL,ITEM)
This procedure inserts an ITEM into a linked queue

1. [Available space?] If AVAIL = NULL, then Write OVERFLOW and Exit
2. [Remove first node from AVAIL list]
   Set NEW := AVAIL and AVAIL := LINK[AVAIL]
3. Set INFO[NEW] := ITEM and LINK[NEW]=NULL
   [Copies ITEM into new node]

Linked Representation of Queues

4. If (FRONT = NULL) then FRONT = REAR = NEW
   [If Q is empty then ITEM is the first element in the queue Q]
   else set LINK[REAR] := NEW and REAR = NEW
   [REAR points to the new node appended to the end of the list]
5. Exit.

Procedure 6.16: LINKQ_DELETE (INFO, LINK, FRONT, REAR, AVAIL, ITEM)
This procedure deletes the front element of the linked queue and stores it in ITEM

1. [Linked queue empty?] if (FRONT = NULL) then Write: UNDERFLOW and Exit
2. Set TEMP = FRONT [If linked queue is nonempty, remember FRONT in a temporary variable TEMP]
3. ITEM = INFO (TEMP)
4. FRONT = LINK (TEMP) [Reset FRONT to point to the next element in the queue]
5. LINK(TEMP) =AVAIL and AVAIL=TEMP [return the deleted node TEMP to the AVAIL list]
6. Exit.

Example- 6.12 [Page- 6.38-6.39]

Deques

Deque (short form of Double-Ended Queue) is a linear list in which elements can be inserted or deleted at either end but not in the middle.

- Pronounced as “DECK” or dequeue

There are various ways of representing a deque in a computer. Unless it is otherwise stated or implied, we will assume our deque is maintained by a circular array DEQUE with pointers LEFT and RIGHT, which point to the two ends of the deque. We assume that the elements extend from the left end to the right end in the array. The term “circular” comes from the fact that we assume that DEQUE[1] comes after DEQUE[N] in the array. Figure 6.26 pictures two deques, each with 4 elements maintained in an array with N = 8 memory locations. The condition LEFT = NULL will be used to indicate that a deque is empty.

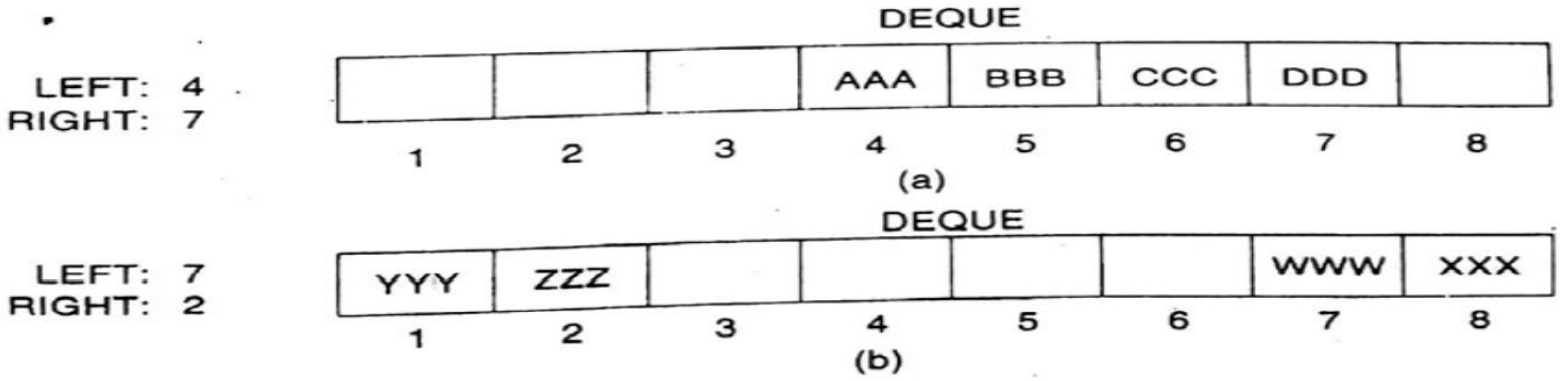


Fig. 6.26

Deques

There are two variations of a deque that are as follows:

1. **Input restricted deque**: It allows insertion of elements at one end only but deletion can be done at both ends.

2. **Output restricted deque**: It allows deletion of elements at one end only but insertion can be done at both ends.

Priority Queues

A **priority queue** is a type of queue in which each element is assigned a **priority** and such that the order in which elements are deleted and processed comes from the following rules :

1. The element with higher priority is processed before any element of lower priority.
2. The elements with the same priority are processed according to the order in which they were added to the queue.

- The elements are inserted at the rear.
- The elements are deleted according to the priority.
- The lower priority number indicates the higher priority.

Applications of Priority Queue:

1. CPU Scheduling
2. Graph algorithms like Dijkstra's shortest path algorithm, Prim's Minimum Spanning Tree, etc.
3. All queue applications where priority is involved.

Array Representation of Priority Queues

A priority queue can be represented in many ways. Here we will discuss two dimensional Array representation of priority queue.

Another way to maintain a priority queue in memory is to use a separate queue for each level of priority (or for each priority number). Each such queue will appear in its own circular array and must have its own pair of pointers, FRONT and REAR. In fact, if each queue is allocated the same amount of space, a two-dimensional array QUEUE can be used instead of the linear arrays. Figure 6.30 indicates this representation for the priority queue in Fig. 6.29. Observe that FRONT[K] and REAR[K] contain, respectively, the front and rear elements of row K of QUEUE, the row that maintains the queue of elements with priority number K.

	FRONT	REAR
1	2	2
2	1	3
3	0	0
4	5	1
5	4	4

	1	2	3	4	5	6
1		AAA				
2	BBB	CCC	XXX			
3						
4	FFF				DDD	EEE
5				GGG		

Array Representation of Priority Queues

The followings are outlines of algorithms for deleting and inserting elements in a priority queue that is maintained in memory by a two-dimensional array QUEUE..

Algorithm 6.19: This algorithm deletes and processes the first element in a priority queue maintained by a two-dimensional array QUEUE.

1. [Find the first nonempty queue.]
   Find the smallest K such that FRONT[K] ≠ NULL.
2. Delete and process the front element in row K of QUEUE.
3. Exit.

Algorithm 6.20: This algorithm adds an ITEM with priority number M to a priority queue maintained by a two-dimensional array QUEUE.

1. Insert ITEM as the rear element in row M of QUEUE.
2. Exit.

One-way List Representation of Priority Queue

One way to maintain a priority queue in memory is by means of a one-way list, as follows:

(a) Each node in the list will contain three items of information: an information field INFO, a priority number PRN and a link number LINK.

(b) A node X precedes a node Y in the list (1) when X has higher priority than Y or (2) when both have the same priority but X was added to the list before Y. This means that the order in the one-way list corresponds to the order of the priority queue.

Priority numbers will operate in the usual way: the lower the priority number, the higher the priority.

Example 6.13

Figure 6.27 shows a schematic diagram of a priority queue with 7 elements. The diagram does not tell us whether BBB was added to the list before or after DDD. On the other hand, the diagram does tell us that BBB was inserted before CCC, because BBB and CCC have the same priority number and BBB appears before CCC in the list. Figure 6.28 shows the way the priority queue may appear in memory using linear arrays INFO, PRN and LINK. (See Sec. 5.2.)

One-way List Representation of Priority Queue

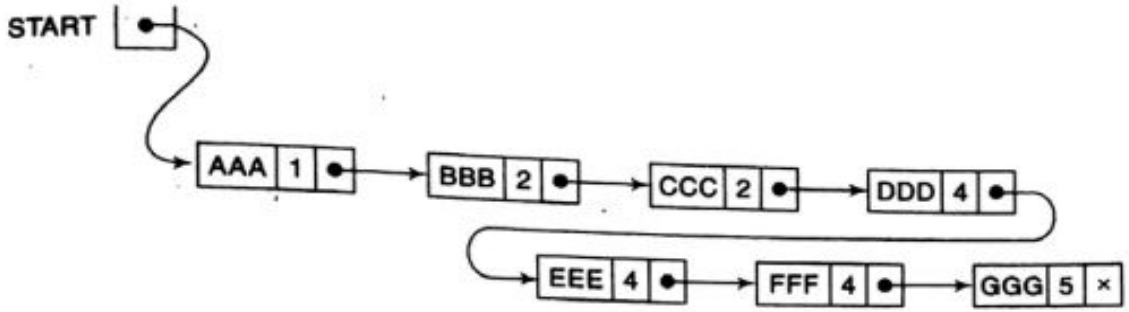


Fig. 6.27

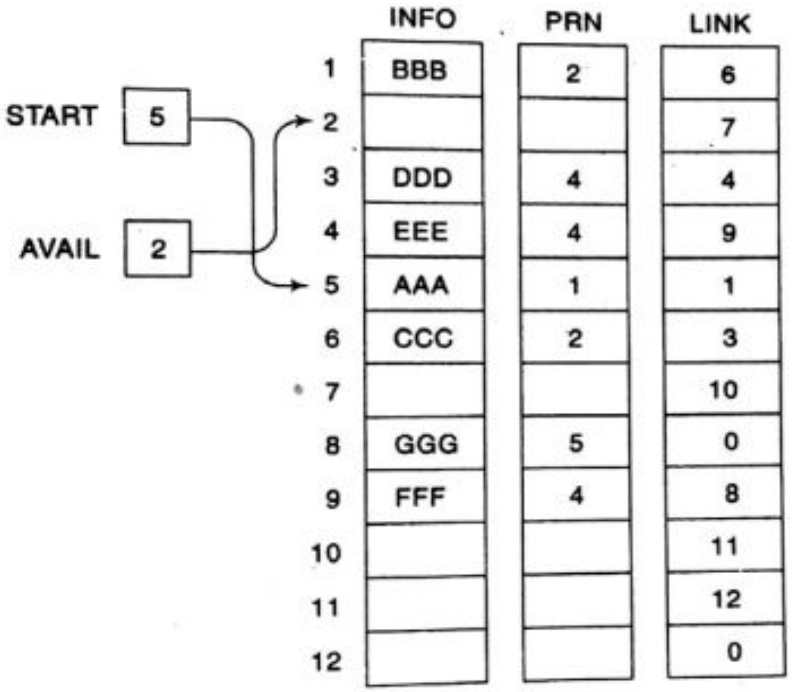


Fig. 6.28

One-way List Representation of Priority Queue

Algorithm 6.17: This algorithm deletes and processes the first element in a priority queue which appears in memory as a one-way list.

1. Set ITEM := INFO[START]. [This saves the data in the first node.]
2. Delete first node from the list.
3. Process ITEM.
4. Exit.

Adding an element to our priority queue is much more complicated than deleting an element from the queue, because we need to find the correct place to insert the element. An outline of the algorithm follows.

Algorithm 6.18: This algorithm adds an ITEM with priority number N to a priority queue which is maintained in memory as a one-way list.

(a) Traverse the one-way list until finding a node X whose priority number exceeds N. Insert ITEM in front of node X.
(b) If no such node is found, insert ITEM as the last element of the list.

One-way List Representation of Priority Queue

processes the first element in a priority queue

Example 6.14

Consider the priority queue in Fig. 6.27. Suppose an item XXX with priority number 2 is to be inserted into the queue. We traverse the list, comparing priority numbers. Observe that DDD is the first element in the list whose priority number exceeds that of XXX. Hence XXX is inserted in the list in front of DDD, as pictured in Fig. 6.29. Observe that XXX comes after BBB and CCC, which have the same priority as XXX. Suppose now that an element is to be deleted from the queue. It will be AAA, the first element in the list. Assuming no other insertions, the next element to be deleted will be BBB, then CCC, then XXX, and so on.

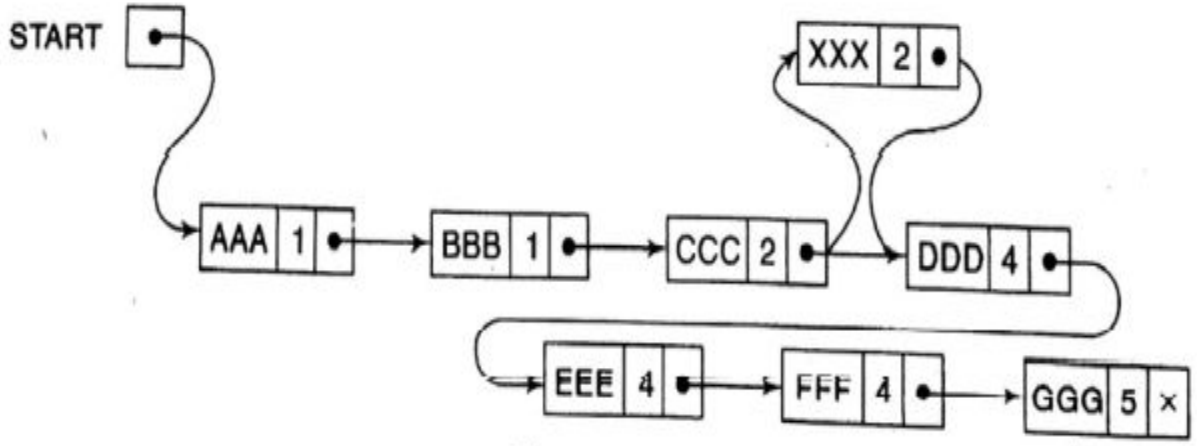


Fig. 6.29

Comparison between Array & One-way List Representation of Priority Queue

Once again we see the time-space tradeoff when choosing between different data structures for a given problem. The array representation of a priority queue is more time-efficient than the one-way list. This is because when adding an element to a one-way list, one must perform a linear search on the list. On the other hand, the one-way list representation of the priority queue may be more space-efficient than the array representation. This is because in using the array representation, overflow occurs when the number of elements in any single priority level exceeds the capacity for that level, but in using the one-way list, overflow occurs only when the total number of elements exceeds the *total* capacity. Another alternative is to use a linked list for each priority level.

Lecture – 12

Sorting

Sorting

- A sorting algorithm is an algorithm that puts elements of a list in a certain order. The most-used orders are **numerical order** and **lexicographical order**.

- **Iterative sorting algorithms (comparison based)**
  - Selection Sort
  - Bubble Sort
  - Insertion Sort
- **Recursive sorting algorithms (comparison based)**
  - Merge Sort
  - Quick Sort
  - Heap Sort
- **Radix sort (non-comparison based)**

- **Given a set of *n* values, there can be *n!* permutations of these values.**
- **O (n log n) is the best possible for any comparison based sorting algorithm which sorts n items.**

Insertion Sort

Suppose an array A with N elements A[1], A[2], ... A[N] is in memory. The insertion sort algorithm scan A from A[1] to A[N], inserting each element A[K] into its proper position in the previously sorting sub array A[1], A[2], ...A[K-1]. That is:

Pass 1: A[1] by itself is trivially sorted

Pass 2: A[2] is inserted either before of after A[1] so that: A[1], A[2] is sorted.

Pass 3: A[3] is inserted into its proper place in A[1], A[2], that is, before A[1}, between A[1] and A[2], or after a[2], so that A[1], A[2], A[3] are sorted.

Pass 4: A[4] is inserted into its proper place in A[1], A[2], A[3] so that: A[1], A[2], A[3], A[4] are sorted.

. . . . . .

Pass N: A[N] is inserted into its proper place in A[1], A[2], ... A[N-1] so that: A[1], A[2], ... A[N] are sorted.

This sorting algorithm is frequently used when N is small. There remains only the problem of deciding how to insert A[K] in its proper place in the subarray A[1], A[2], ... A[K-1]. This can be accomplished by comparing A[K] with A[K-1], comparing A[K] with A[K-2], comparing A[K] with A[K-3], and so on, until first meeting an element A[J] such that A[J] ≤ A[K]. Then each of elements A[K-1], A[K-2], ... A[J+1] is moved forward one location, and A[K] is then inserted in the J+1 st position in the array.

Insertion Sort

Suppose an array A contains 8 elements as follows:

77, 33, 44, 11, 88, 22, 66, 55

Figure 9.3 illustrates the insertion sort algorithm. The circled element indicates the A[K] in each pass of the algorithm, and the arrow indicates the proper place for inserting A[K].

Pass	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K = 1:	−∞	(77)	33	44	11	88	22	66	55
K = 2:	−∞	77	(33)	44	11	88	22	66	55
K = 3:	−∞	33	77	(44)	11	88	22	66	55
K = 4:	−∞	33	44	77	(11)	88	22	66	55
K = 5:	−∞	11	33	44	77	(88)	22	66	55
K = 6:	−∞	11	33	44	77	88	(22)	66	55
K = 7:	−∞	11	22	33	44	77	88	(66)	55
K = 8:	−∞	11	22	33	44	66	77	88	(55)
Sorted:	−∞	11	22	33	44	55	66	77	88

Fig. 9.3 *Insertion Sort for n = 8 Items*

Insertion Sort

Algorithm 9.1: (Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements.

1. Set A[0] := $-\infty$. [Initializes sentinel element.]
2. Repeat Steps 3 to 5 for K = 2, 3, ..., N:
3. Set TEMP := A[K] and PTR := K – 1.
4. Repeat while TEMP < A[PTR]:
   (a) Set A[PTR + 1] := A[PTR]. [Moves element forward.]
   (b) Set PTR := PTR – 1.
   [End of loop.]
5. Set A[PTR + 1] := TEMP. [Inserts element in proper place.]
   [End of Step 2 loop.]
6. Return.

Insertion Sort

Complexity of Insertion Sort

The number $f(n)$ of comparisons in the insertion sort algorithm can be easily computed. First of all, the worst case occurs when the array A is in reverse order and the inner loop must use the maximum number K – 1 of comparisons. Hence

$$f(n) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = O(n^2)$$

Furthermore, one can show that, on the average, there will be approximately (K – 1)/2 comparisons in the inner loop. Accordingly, for the average case,

$$f(n) = \frac{1}{2} + \frac{2}{2} + \cdots + \frac{n-1}{2} = \frac{n(n-1)}{4} = O(n^2)$$

Thus the insertion sort algorithm is a very slow algorithm when n is very large.

The above results are summarized in the following table:

Algorithm	*Worst Case*	*Average Case*
Insertion Sort	$\frac{n(n-1)}{2} = O(n^2)$	$\frac{n(n-1)}{4} = O(n^2)$

Selection Sort

Suppose an array A with n elements A[1], A[2], ..., A[N] is in memory. The selection sort algorithm for sorting A works as follows. First find the smallest element in the list and put it in the first position. Then find the second smallest element in the list and put it in the second position. And so on. More precisely:

Pass 1. Find the location LOC of the smallest in the list of N elements
A[1], A[2], ..., A[N], and then interchange A[LOC] and A[1]. Then: A[1] is sorted.

Pass 2. Find the location LOC of the smallest in the sublist of N – 1 elements
A[2], A[3], ..., A[N], and then interchange A[LOC] and A[2]. Then:
A[1], A[2] is sorted, since A[1] ≤ A[2].

Pass 3. Find the location LOC of the smallest in the sublist of N – 2 elements
A[3], A[4], ..., A[N], and then interchange A[LOC] and A[3]. Then:
A[1], A[2], ..., A[3] is sorted, since A[2] ≤ A[3].

... .

... .

Pass N – 1. Find the location LOC of the smaller of the elements A[N – 1], A[N], and then interchange A[LOC] and A[N – 1]. Then:
A[1], A[2], ..., A[N] is sorted, since A[N – 1] ≤ A[N].

Thus A is sorted after N – 1 passes.

Selection Sort

Suppose an array A contains 8 elements as follows:

77, 33, 44, 11, 88, 22, 66, 55

Applying the selection sort algorithm to A yields the data in Fig. 9.4. Observe that LOC gives the location of the smallest among A[K], A[K + 1], ..., A[N] during Pass K. The circled elements indicate the elements which are to be interchanged.

Pass	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K = 1, LOC = 4	(77)	33	44	(11)	88	22	66	55
K = 2, LOC = 6	11	(33)	44	77	88	(22)	66	55
K = 3, LOC = 6	11	22	(44)	77	88	(33)	66	55
K = 4, LOC = 6	11	22	33	(77)	88	(44)	66	55
K = 5, LOC = 8	11	22	33	44	(88)	77	66	(55)
K = 6, LOC = 7	11	22	33	44	55	(77)	(66)	88
K = 7, LOC = 7	11	22	33	44	55	66	(77)	88
Sorted:	11	22	33	44	55	66	77	88

Fig. 9.4 *Selection Sort for n = 8 Items*

Selection Sort

Procedure 9.2: MIN(A, K, N, LOC)
An array A is in memory. This procedure finds the location LOC of the smallest element among A[K], A[K + 1], ..., A[N].

1. Set MIN := A[K] and LOC := K. [Initializes pointers.]
2. Repeat for J = K + 1, K + 2, ..., N:
   If MIN > A[J], then: Set MIN := A[J] and LOC := A[J] and LOC := J.
   [End of loop.]
3. Return.

The selection sort algorithm can now be easily stated:

Algorithm 9.3: (Selection Sort) SELECTION(A, N)
This algorithm sorts the array A with N elements.

1. Repeat Steps 2 and 3 for K = 1, 2, ..., N – 1:
2. Call MIN(A, K, N, LOC).
3. [Interchange A[K] and A[LOC].]
   Set TEMP := A[K], A[K] := A[LOC] and A[LOC] := TEMP.
   [End of Step 1 loop.]
4. Exit.

Selection Sort

Complexity of the Selection Sort Algorithm

First note that the number $f(n)$ of comparisons in the selection sort algorithm is independent of the original order of the elements. Observe that MIN(A, K, N, LOC) requires $n - K$ comparisons. That is, there are $n - 1$ comparisons during Pass 1 to find the smallest element, there are $n - 2$ comparisons during Pass 2 to find the second smallest element, and so on. Accordingly,

$$f(n) = (n - 1) + (n - 2) + \cdots + 2 + 1 = \frac{n(n-1)}{2} = O(n^2)$$

The above result is summarized in the following table:

Algorithm	*Worst Case*	*Average Case*
Selection Sort	$\frac{n(n-1)}{2} = O(n^2)$	$\frac{n(n-1)}{2} = O(n^2)$

Remark: The number of interchanges and assignments does depend on the original order of the elements in the array A, but the sum of these operations does not exceed a factor of n^2.

Merging

Suppose A is a sorted list with r elements and B is a sorted list with s elements. The operation that combines the elements of A and B into a single sorted list C with $n = r + s$ elements is called *merging*. One simple way to merge is to place the elements of B after the elements of A and then use some sorting algorithm on the entire list. This method does not take advantage of the fact that A and B are individually sorted. A much more efficient algorithm is Algorithm 9.4 in this section. First, however, we indicate the general idea of the algorithm by means of two examples.

Suppose one is given two sorted decks of cards. The decks are merged as in Fig. 9.5. That is, at each step, the two front cards are compared and the smaller one is placed in the combined deck. When one of the decks is empty, all of the remaining cards in the other deck are put at the end of the combined deck. Similarly, suppose we have two lines of students sorted by increasing heights, and suppose we want to merge them into a single sorted line. The new line is formed by choosing, at each step, the shorter of the two students who are at the head of their respective lines. When one of the lines has no more students, the remaining students line up at the end of the combined line.

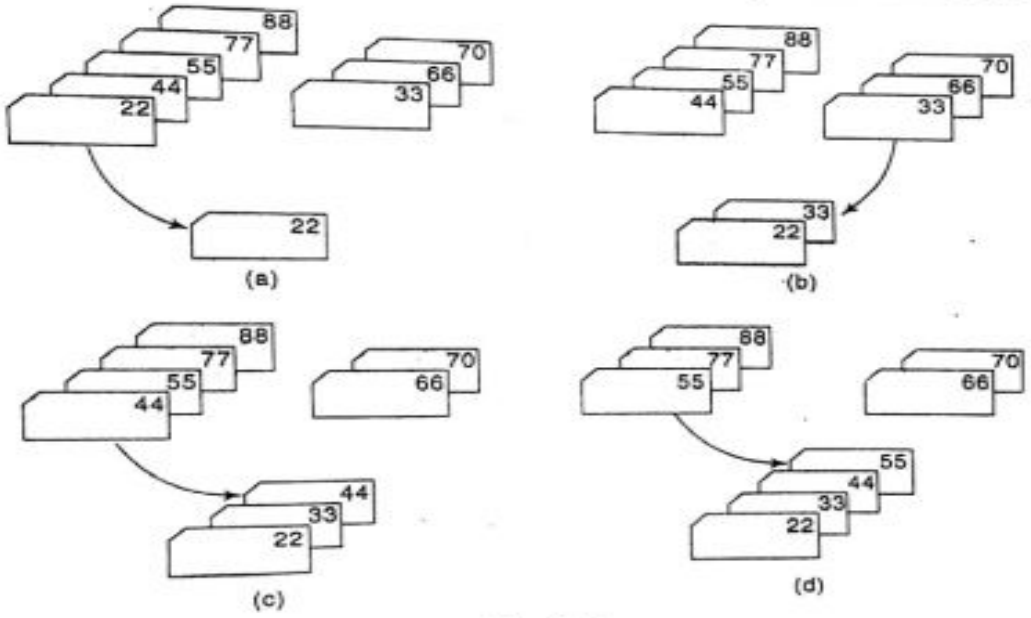


Fig. 9.5

Merging

Algorithm 9.4: MERGING(A, R, B, S, C)

Let A and B be sorted arrays with R and S elements, respectively. This algorithm merges A and B into an array C with N = R + S elements.

1. [Initialize.] Set NA := 1, NB := 1 and PTR := 1.
2. [Compare.] Repeat while NA ≤ R and NB ≤ S:
   If A[NA] < B[NB], then:
   (a) [Assign element from A to C.] Set C[PTR] := A[NA].
   (b) [Update pointers.] Set PTR := PTR + 1 and NA := NA + 1.
   Else:
   (a) [Assign element from B to C.] Set C[PTR] := B[NB].
   (b) [Update pointers.] Set PTR := PTR + 1 and NB := NB + 1.
   [End of If structure.]
   [End of loop.]
3. [Assign remaining elements to C.]
   If NA > R, then:
   Repeat for K = 0, 1, 2, ..., S – NB:
   Set C[PTR + K] := B[NB + K].
   [End of loop.]
   Else:
   Repeat for K = 0, 1, 2, ..., R – NA:
   Set C[PTR + K] := A[NA + K].
   [End of loop.]
   [End of If structure.]
4. Exit.

Merge Sort

- Suppose an array A with n elements A[1], A[2], …., A[N] is in memory. The merge-sort algorithm which sorts A will first be described by means of a specific example.

Example 9.7

Suppose the array A contains 14 elements as follows:

66, 33, 40, 22, 55, 88, 60, 11, 80, 20, 50, 44, 77, 30

Each pass of the merge-sort algorithm will start at the beginning of the array A and merge pairs of sorted subarrays as follows:

Pass 1. Merge each pair of elements to obtain the following list of sorted pairs:

33, 66 22, 40 55, 88 11, 60 20, 80 44, 50 30, 70

Pass 2. Merge each pair of pairs to obtain the following list of sorted quadruplets:

22, 33, 40, 66 11, 55, 60, 88 20, 44, 50, 80 30, 77

Pass 3. Merge each pair of sorted quadruplets to obtain the following two sorted subarrays:

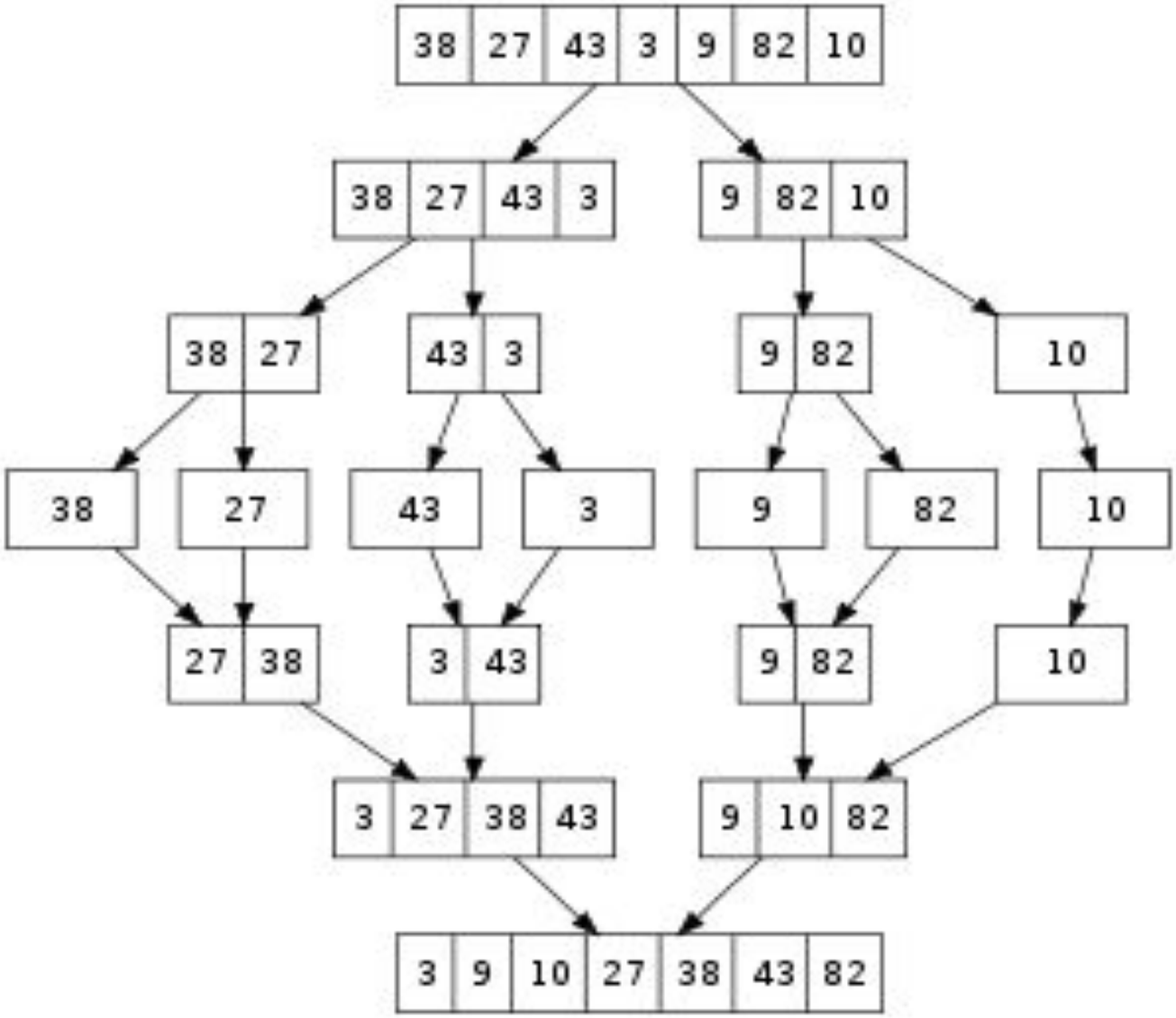
11, 22, 33, 40, 55, 60, 66, 88 20, 30, 44, 50, 77, 80

Pass 4. Merge the two sorted subarrays to obtain the single sorted array

11, 20, 22, 30, 33, 40, 44, 50, 55, 60, 66, 77, 80, 88

The original array A is now sorted.

Merge Sort


38 27 43 3 9 82 10
38 27 43 3
9 82 10
38 27
43 3
9 82
10
38
27
43
3
9
82
10
27 38
3 43
9 82
10
3 27 38 43
9 10 82
3 9 10 27 38 43 82

Merge Sort

Procedure 9.6: MERGEPASS(A, N, L, B)
The N-element array A is composed of sorted subarrays where each subarray has L elements except possibly the last subarray, which may have fewer than L elements. The procedure merges the pairs of subarrays of A and assigns them to the array B.

1. Set Q := INT(N/(2*L)), S := 2*L*Q and R := N – S.
2. [Use Procedure 9.5 to merge the Q pairs of subarrays.]
   Repeat for J = 1, 2, ..., Q:
   (a) Set LB := 1 + (2*J – 2)*L. [Finds lower bound of first array.]
   (b) Call MERGE(A, L, LB, A, L, LB + L, B, LB).
   [End of loop.]
3. [Only one subarray left?]
   If R ≤ L, then:
   Repeat for J = 1, 2, ..., R:
   Set B(S + J) := A(S + J).
   [End of loop.]
   Else:
   Call MERGE(A, L, S + 1, A, R, L + S + 1, B, S + 1).
   [End of If structure.]
4. Return.

Algorithm 9.7: MERGESORT(A, N)
This algorithm sorts the N-element array A using an auxiliary array B.

1. Set L := 1. [Initializes the number of elements in the subarrays.]
2. Repeat Steps 3 to 6 while L < N:
3. Call MERGEPASS(A, N, L, B).
4. Call MERGEPASS(B, N, 2 * L, A).
5. Set L := 4 * L.
   [End of Step 2 loop.]
6. Exit.

Merge Sort

Complexity of the Merge-Sort Algorithm

Let $f(n)$ denote the number of comparisons needed to sort an n-element array A using the merge-sort algorithm. Recall that the algorithm requires at most $\log n$ passes. Moreover, each pass merges a total of n elements, and by the discussion on the complexity of merging, each pass will require at most n comparisons. Accordingly, for both the worst case and average case,

$$f(n) \leq n \log n$$

Observe that this algorithm has the same order as heapsort and the same average order as quicksort. The main drawback of merge-sort is that it requires an auxiliary array with n elements. Each of the other sorting algorithms we have studied requires only a finite number of extra locations, which is independent of n.

The above results are summarized in the following table:

Algorithm	*Worst Case*	*Average Case*	*Extra Memory*
Merge-Sort	$n \log n = O(n \log n)$	$n \log n = O(n \log n)$	$O(n)$

Quicksort

- Quick sort is a divide and conquer algorithm.
- Quick sort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quick sort can then recursively sort the sub-lists.
- The steps are:

1. Pick an element, called a pivot, from the list.
2. Reorder the list so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the partition operation.
3. Recursively apply the above steps to the sub-list of elements with smaller values and separately the sub-list of elements with greater values.

The base case of the recursion are lists of size zero or one, which never need to be sorted.

Quick Sort

Suppose A is the following list of 12 numbers:

44, 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, 66

The reduction step of the quicksort algorithm finds the final position of one of the numbers; in this illustration, we use the first number, 44. This is accomplished as follows. Beginning with the last number, 66, scan the list from right to left, comparing each number with 44 and stopping at the first number less than 44. The number is 22. Interchange 44 and 22 to obtain the list

22, 33, 11, 55, 77, 90, 40, 60, 99, 44, 88, 66

(Observe that the numbers 88 and 66 to the right of 44 are each greater than 44.) Beginning with 22, next scan the list in the opposite direction, from left to right, comparing each number with 44 and stopping at the first number greater than 44. The number is 55. Interchange 44 and 55 to obtain the list

22, 33, 11, 44, 77, 90, 40, 60, 99, 55, 88, 66

(Observe that the numbers 22, 33 and 11 to the left of 44 are each less than 44.) Beginning this time with 55, now scan the list in the original direction, from right to left, until meeting the first number less than 44. It is 40. Interchange 44 and 40 to obtain the list

22, 33, 11, 40, 77, 90, 44, 60, 99, 55, 88, 66

(Again, the numbers to the right of 44 are each greater than 44.) Beginning with 40, scan the list from left to right. The first number greater than 44 is 77. Interchange 44 and 77 to obtain the list

22, 33, 11, 40, 44, 90, 77, 60, 99, 55, 88, 66

(Again, the numbers to the left of 44 are each less than 44.) Beginning with 77, scan the list from right to left seeking a number less than 44. We do not meet such a number before meeting 44. This means all numbers have been scanned and compared with 44. Furthermore, all numbers less than 44 now form the sublist of numbers to the left of 44, and all numbers greater than 44 now form the sublist of numbers to the right of 44, as shown below:

22, 33, 11, 40, 44, 90, 77, 60, 99, 55, 88, 66

First sublist Second sublist

Thus 44 is correctly placed in its final position, and the task of sorting the original list A has now been reduced to the task of sorting each of the above sublists.

Quick Sort

Complexity of the Quicksort Algorithm

The running time of a sorting algorithm is usually measured by the number $f(n)$ of comparisons required to sort n elements. The quicksort algorithm, which has many variations, has been studied extensively. Generally speaking, the algorithm has a worst-case running time of order $n^2/2$, but an average-case running time of order $n \log n$. The reason for this is indicated below.

The worst case occurs when the list is already sorted. Then the first element will require n comparisons to recognize that it remains in the first position. Furthermore, the first sublist will be empty, but the second sublist will have $n - 1$ elements. Accordingly, the second element will require $n - 1$ comparisons to recognize that it remains in the second position. And so on. Consequently, there will be a total of

$$f(n) = n + (n-1) + \cdots + 2 + 1 = \frac{n(n+1)}{2} = \frac{n^2}{2} + O(n) = O(n^2)$$

comparisons. Observe that this is equal to the complexity of the bubble sort algorithm (Sec. 4.6).

The complexity $f(n) = O(n \log n)$ of the average case comes from the fact that, on the average, each reduction step of the algorithm produces two sublists. Accordingly:

(1) Reducing the initial list places 1 element and produces two sublists.
(2) Reducing the two sublists places 2 elements and produces four sublists.
(3) Reducing the four sublists places 4 elements and produces eight sublists.
(4) Reducing the eight sublists places 8 elements and produces sixteen sublists.

And so on. Observe that the reduction step in the kth level finds the location of 2^{k-1} elements; hence there will be approximately $\log_2 n$ levels of reductions steps. Furthermore, each level uses at most n comparisons, so $f(n) = O(n \log n)$. In fact, mathematical analysis and empirical evidence have both shown that

$$f(n) \approx 1.4 \lceil n \log n \rceil$$

is the expected number of comparisons for the quicksort algorithm.

Quick Sort

```
// C qsort and C++ sort() algorithm
#include <bits/stdc++.h>
using namespace std;
#define N 100
// A comparator function used by qsort
int compare(const void * a, const void * b)
{
        return ( *(int*)a - *(int*)b );
}
int main()
{
        int arr[N+1], Arr2[N+1];
        srand(time(NULL));
        // generate random input
        for (int i = 0; i < N; i++)
                Arr2[i] = arr[i] = rand()%100000;
        qsort(arr, N, sizeof(int), compare);
        for (int i = 0; i < N; i++)
                cout<< arr[i] << " ";
        sort(Arr2, Arr2 + N);
        cout<<endl;
        for (int i = 0; i < N; i++)
                cout<< Arr2[i] << " ";
        return 0;
}
```

Sorting Algorithms

Algorithm	Worst Case	Average Case	Extra Memory	Stable?
Insertion sort	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection sort	$O(n^2)$	$O(n^2)$	O(1)	No
Bubble sort	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge sort	$O(n\ lgn)$	$O(n\ logn)$	O(n)	Yes
Quick sort	$O(n^2)$	$O(n\ logn)$	O(1)	No
Heap sort	$O(n\ lgn)$	$O(n\ logn)$	O(1)	No

- **A sort algorithm is said to be an in-place sort, If it requires only a constant amount (i.e. O(1)) of extra space during the sorting process.**

- **A sorting algorithm is stable if the relative order of elements with the same key value is preserved by the algorithm.**

Lecture – 13

Hashing

Searching & Data Modification

Data modification refers to the operations of inserting, deleting and updating. Here data modification will mainly refer to inserting and deleting. These operations are closely related to searching, since usually one must search for the location of the ITEM to be deleted or one must search for the proper place to insert ITEM in the table. The insertion or deletion also requires a certain amount of execution time, which also depends mainly on the type of data structure that is used.

Generally speaking, there is a tradeoff between data structures with fast searching algorithms and data structures with fast modification algorithms. This situation is illustrated below, where we summarize the searching and data modification of three of the data structures previously studied in the text.

(1) *Sorted array.* Here one can use a binary search to find the location LOC of a given ITEM in time $O(\log n)$. On the other hand, inserting and deleting are very slow, since, on the average, $n/2 = O(n)$ elements must be moved for a given insertion or deletion. Thus a sorted array would likely be used when there is a great deal of searching but only very little data modification.

(2) *Linked list.* Here one can only perform a linear search to find the location LOC of a given ITEM, and the search may be very, very slow, possibly requiring time $O(n)$. On the other hand, inserting and deleting requires only a few pointers to be changed. Thus a linked list would be used when there is a great deal of data modification, as in word (string) processing.

(3) *Binary search tree.* This data structure combines the advantages of the sorted array and the linked list. That is, searching is reduced to searching only a certain path P in the tree T, which, on the average, requires only $O(\log n)$ comparisons. Furthermore, the tree T is maintained in memory by a linked representation, so only certain pointers need be changed after the location of the insertion or deletion is found. The main drawback of the binary search tree is that the tree may be very unbalanced, so that the length of a path P may be $O(n)$ rather than $O(\log n)$. This will reduce the searching to approximately a linear search.

Hashing

- **Hashing** is a searching technique, which is essentially independent of the number n of input data.

- The idea of hashing can be introduced by the following example.

  Suppose a company with 68 employees assigns a 4-digit employee number to each employee which is used as the primary key in the company‘s employee file. We can, in fact, use the employee number as the address of the record in memory. The search will require no comparisons at all. Unfortunately, this technique will require space for 10,000 memory locations, whereas space for fewer than 30 such locations would actually be used. Clearly, this tradeoff of space for time is not worth the expense.

- The general idea of using the key to determine the address of a record is an excellent idea, but it must be modified so that a great deal of space is not wasted.
- This modification takes the form of a function H from the set K of keys into the set L of memory addresses. Such a function, H: K → L is called a **hash function** or **hashing function.**

Hash Table

- **Hash table** is a data structure used for storing and retrieving data quickly. Insertion of data in the hash table is based on the key value. Hence every entry in the hash table is associate with some key. For example, for an employee record in the hash table employee ID will works as a key.
- There are three components that are involved with performing storage and retrieval with Hash Tables:
  - **A hash table**: This is a fixed size table that stores data of a given type.
  - **A hash function**: This is a function that converts a piece of data into an integer. Sometimes we call this integer a **hash value**. The integer should be at least as big as the hash table. When we store a value in a hash table, we compute its hash value with the hash function, take that value modulo the hash table size, and that's where we store/retrieve the data.
  - **A collision resolution strategy**: There are times when two pieces of data have hash values that, when taken modulo the hash table size, yield the same value. That is called a collision. We need to handle collisions.

Hash Function

- Hash function is a function which is used to put data into hash table. Hence one can use the same as function to retrieve the data from hash table. Thus hash function is used to implement a hash table.
- The two principal criteria used in selecting a hash function H: K → L are as follows.
  - First of all, the function H should be very easy and quick to compute.
  - Secondly the function H should, as far as possible, uniformly distribute the hash addresses throughout the set L so that there are a minimum number of collisions.
    - Naturally, there is no guarantee that the second condition can be completely fulfilled without actually knowing beforehand the keys and addresses. However, certain general techniques do help.
- Some popular hash functions are :

  a) Division hash function method

  b) Mid square hash function method

  c) Digit folding or folding hash function method

Hash Function

a) Division method

Choose a number **m** larger than the number **n** of keys in **K.** (The number m is usually chosen to be a **prime number** or a number without small divisors, since this frequently minimizes the number of collisions.) The hash functions H is defined by

H(k) = k(mod m) or H(k) = k(mod m) + 1

Here k (mod m) denotes the remainder when k is divided by m.

The second formula is used when we want the hash addresses to range from 1 to m rather than from 0 to m-1.

b) Midsquare method

The key k is squared. Then the hash function H is defined by

H(k) = l

Where l is obtained by deleting digits from both ends of k^2.

c) Folding method:

The key k is partitioned into a number of parts, k_1, k_2, ..., k_r, where each part, except possibly the last, has the same number of digits as the required address. Then the parts are added together, ignoring the last carry. That is,

$H(k) = k_1+k_2+.....+k_r$

Where the leading-digit carries, if any, are ignored.

Sometimes, for extra - ”milling”, the even-numbered parts, k_2,k_4,....., are each reversed before the addition.

Hash Function

Consider the company in Example 9.9, each of whose 68 employees is assigned a unique 4-digit employee number. Suppose L consists of 100 two-digit addresses: 00, 01, 02, ..., 99. We apply the above hash functions to each of the following employee numbers:

3205, 7148, 2345

(a) *Division method.* Choose a prime number m close to 99, such as $m = 97$. Then

$$H(3205) = 4, \quad H(7148) = 67, \quad H(2345) = 17$$

That is, dividing 3205 by 97 gives a remainder of 4, dividing 7148 by 97 gives a remainder of 67, and dividing 2345 by 97 gives a remainder of 17. In the case that the memory addresses begin with 01 rather than 00, we choose that the function $H(k) = k \pmod m + 1$ to obtain:

$$H(3205) = 4 + 1 = 5, \quad H(7148) = 67 + 1 = 68, \quad H(2345) = 17 + 1 = 18$$

(b) *Midsquare method.* The following calculations are performed:

k:	3205	7148	2345
k^2:	10 272 025	51 093 904	5 499 025
$H(k)$:	72	93	99

Observe that the fourth and fifth digits, counting from the right, are chosen for the hash address.

(c) *Folding method.* Chopping the key k into two parts and adding yields the following hash addresses:

$$H(3205) = 32 + 05 = 37, \quad H(7148) = 71 + 48 = 19, \quad H(2345) = 23 + 45 = 68$$

Observe that the leading digit 1 in $H(7148)$ is ignored. Alternatively, one may want to reverse the second part before adding, thus producing the following hash addresses:

$$H(3205) = 32 + 50 = 82, \quad H(7148) = 71 + 84 + 55, \quad H(2345) = 23 + 54 = 77$$

Collision Resolution

- Suppose we want to add a new record R with key k to our file F, but suppose the memory location address H(k) is already occupied. This situation is called **collision**.
- If collision occurs then it should be handled by applying some techniques. Such techniques are called **collision resolution technique**.
- The goal of collision resolution techniques is to minimize collisions. There are two methods of handling collisions.
  1. **Open hashing** or **Separate Chaining**
  2. **Closed hashing** or **Open addressing**
- The difference between open hashing and closed hashing is that in Open hashing the collision are stored outside table and in Closed hashing the collisions are stored in the same table at some another slot.

Closed hashing or Open addressing

- In Closed hashing the collisions are stored in the same table at some another slot. For Closed hashing one of the following technique is adopted.
  1. Linear probing
  2. Quadratic probing
  3. Double probing or Double hashing

Linear probing

- Suppose that a new record R with a key k is to be added to the memory table T, but that the memory location with hash address H(k)=h is already filled. One natural way to resolve the collision is to assign R to the first available location following T[h]. (We assume that the table t with m locations is circular, so that T[1] comes after T[m].) Accordingly, with such a collision procedure, we will search for the record R in the table T by linearly searching the locations T[h], T[h+1], T[h+2],.....until finding R or meeting an empty location, which indicates an unsuccessful search. The above collision resolution is called **linear probing**.
- One main disadvantage of linear probing is that records tend to **cluster**, that is, appear next to one another.

Closed hashing or **Open addressing**

Example:

Consider that following keys are to be inserted in the hash table:

131, 4, 8, 7, 21, 5, 31, 61, 9, 29.

The hash table size is 10. We will use division hash function i.e.

H (Key) = key % table size

For instance the element 131 can be placed at H (Key) = 131 % 10 =1.

0	29
1	131
2	21
3	31
4	4
5	5
6	61
7	7
8	8
9	9

Class work: Draw the table when H (Key) = key % 11 + 1

Closed hashing or Open addressing

Quadratic probing

Suppose a record R with key k has the hash address H(k) = h. Then, instead of searching the locations with addresses h, h+1, h+2,....., we linearly search the locations with addresses h, h+1, h+4, h+9, h+16,.....h+i^2,.....

If the number m of locations in the table T is a prime number, then the above sequence will access half of the locations in T.

This method uses following formula:

H(Key) = (H (Key) + i^2) % m

Where 'm' can be table size or any prime number.

Example: -

Insert following elements in the hash table with table size 10.

37, 19, 55, 22, 17, 49, 87.

0	49
1	87
2	22
3	
4	
5	55
6	
7	37
8	17
9	19

Closed hashing or Open addressing

Double hashing

Here a second hash function H‘ is used for resolving a collision, as follows. Suppose a record R with key k has the hash addresses H(k) = h and H‘(k) = h‘ ≠ m. Then we linearly search the locations with addresses h, h+h‘, h+2h‘, h+3h‘,....

If m is a prime number, then the above sequence will access all the locations in the table T.

This method uses following formula:

H1 (key) = k % table size

A popular second hash function is:

H2 (key) = M - (K % M)

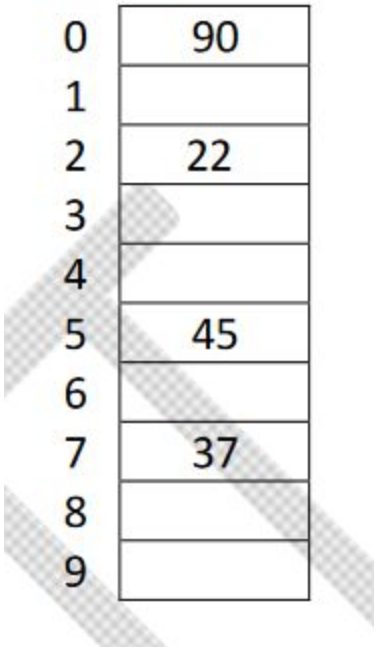
Where M is prime number smaller than the size of the table.

H(Key) = (H1 + i. H2) % table size

Example: consider the following elements to be placed in the Hash table of size 10.

37, 90, 45, 22, 17, 49, 55.

Now find where 17 will be inserted.



Closed hashing or **Open addressing**

Remark: One major disadvantage in any type of open addressing procedure is in the implementation of deletion. Specifically, suppose a record R is deleted from the location T[r]. Afterwards, suppose we meet T[r] while searching for another record R‘. This does not necessarily mean that the search is unsuccessful. Thus, when deleting the record R, we must label the location T[r] to indicate that it previously did contain a record. Accordingly, open addressing may seldom be used when a file F is constantly changing.

Open hashing or Separate **Chaining**

Open Hashing, is a technique in which the data is not directly stored at the hash key index (k) of the Hash table. Rather the data at the key index (k) in the hash table is a pointer to the head of the data structure where the data is actually stored. In the most simple and common implementations the data structure adopted for storing the element is a linked-list.

In this technique when a data needs to be searched, it might become necessary (worst case) to traverse all the nodes in the linked list to retrieve the data.

Example:-

Consider the keys to be placed in the home buckets are

131, 3, 4, 21, 61, 24, 7, 97, 8, 9.

A chain is maintained for colliding elements. For example 131 has a home bucket index 1. Similarly keys 21 and 61 demand for home bucket index 1. Hence a chain is maintained at index 1. Similarly the chain at index 4 and 7 is maintained.

Separate Chaining Vs Open Addressing

Separate Chaining	Open Addressing
Keys are stored inside the hash table as well as outside the hash table.	All the keys are stored only inside the hash table. No key is present outside the hash table.
The number of keys to be stored in the hash table can even exceed the size of the hash table.	The number of keys to be stored in the hash table can never exceed the size of the hash table.
Deletion is easier.	Deletion is difficult.
Extra space is required for the pointers to store the keys outside the hash table.	No extra space is required.
Cache performance is poor. This is because of linked lists which store the keys outside the hash table.	Cache performance is better. This is because here no linked lists are used.
Some buckets of the hash table are never used which leads to wastage of space.	Buckets may be used even if no key maps to those particular buckets.

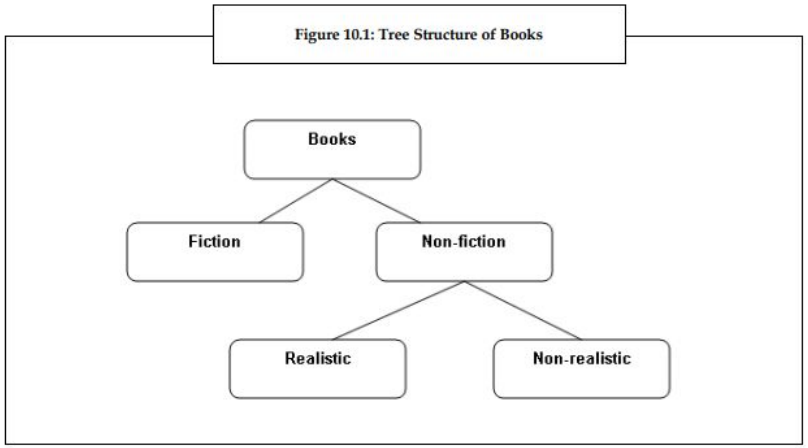
Lecture – 14

Tree

Tree

A tree is a very important **non linear data structure** as it is useful in many applications. A tree structure is a method of representing the **hierarchical** nature of a structure in a graphical form. It is termed as "tree structure" since its representation resembles a tree. However, the chart of a tree is normally upside down compared to an actual tree, with the **root** at the *top* and the **leaves** at the *bottom*.

The figure 10.1 depicts a tree structure, which represents the hierarchical organization of **books**.



In the hierarchical organization of books shown in figure 10.1, Books is the **root** of the tree. Books can be classified as Fiction and Non-fiction. Non-fiction books can be further classified as Realistic and Non-realistic, which are the **leaves** of the tree. Thus, it forms a complete tree structure.

Binary Tree

- A **binary tree** is a tree in which each node can have ***at most two children***.
- A binary tree is either *empty* or consists of a node called the *root* together with two binary trees called the *left subtree* and the *right subtree*.
- A tree with no nodes is called as a *null tree*.

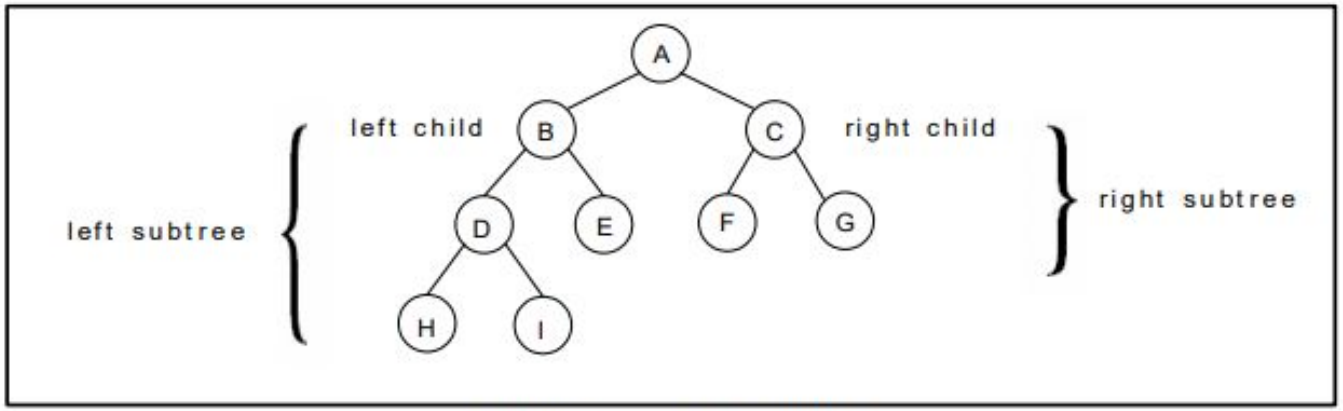


Figure . Binary Tree

- A Binary Tree T is defined as finite set of elements, called nodes, such that:

  a) T is empty (called the null tree or empty tree.)

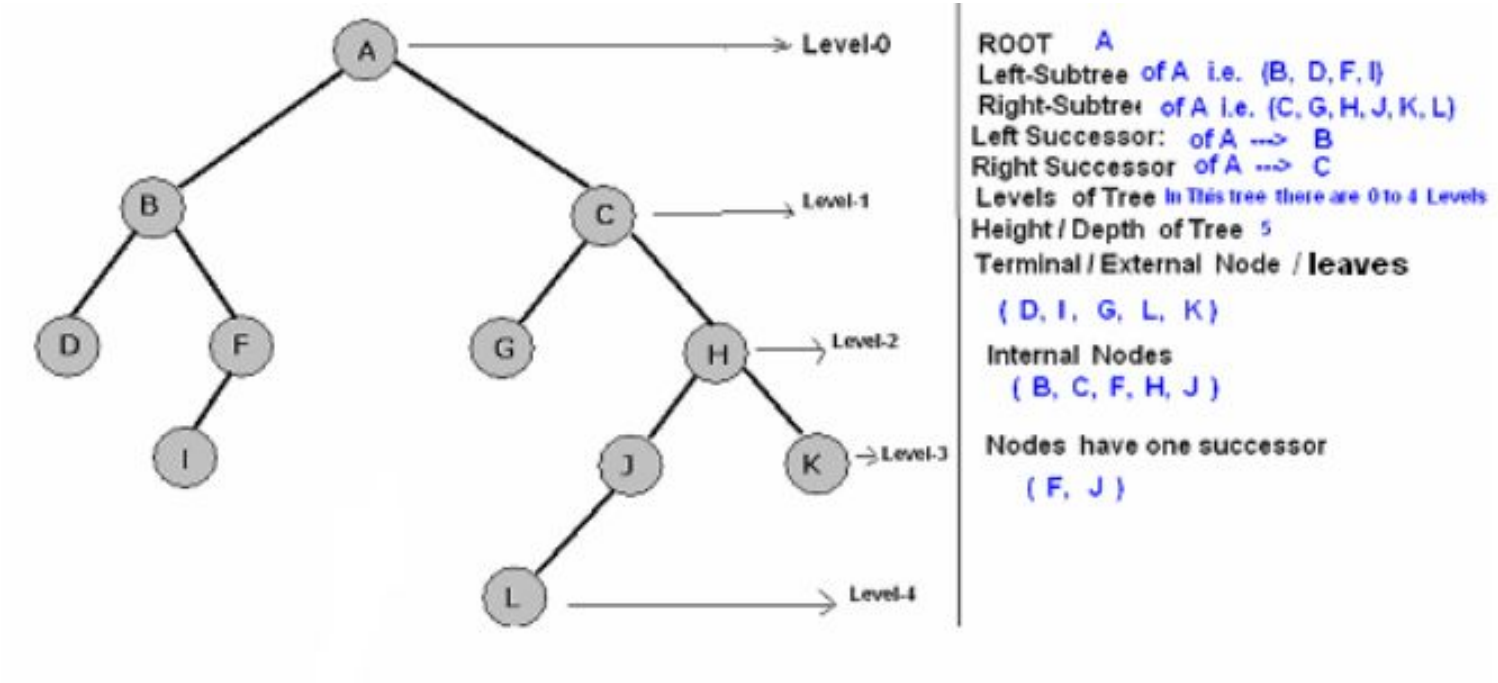
  b) T contains a distinguished node R, called the **root** of T, and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2.

If T does contain a root R, then the two trees T_1 and T_2 are called, respectively, the *left sub tree* and *right sub tree* of R. If T_1 is non empty, then its root is called the *left successor* of R; similarly, if T_2 is non empty, then its root is called the *right successor* of R. The nodes with no successors are called the **terminal** nodes.

Tree Terminology

- If N is a node in T with left successor S_1 and right successor S_2, then N is called the **parent**(or father) of S_1 and S_2. Analogously, S_1 is called the **left child** (or son) of N, and S_2 is called the **right child** (or son) of N. Furthermore, S_1 and S_2 are said to **siblings** (or brothers). Every node in the binary tree T, except the root, has a *unique* parent, called the **predecessor** of N.
- The terms descendant and ancestor have their usual meaning. A node L is called a **descendant** of a node N (and N is called an **ancestor** of L) if there is a succession of children from N to L.
- The line drawn from a node N of T to a successor is called an **edge**, and a sequence of consecutive edges is called a **path**. A terminal node is called a **leaf**, and a path ending in a leaf is called a **branch**.
- Each node in binary tree T is assigned a **level number**, as follows. The root R of the tree T is assigned the level number 0, and every other node is assigned a level number which is 1 more than the level number of its parent. *The maximum number of nodes at any level is* 2^n. Those nodes with the same level number are said to belong to the **same generation**.
- The **depth** (or **height**) of a tree T is the maximum number of nodes in a branch of T. This turns out to be 1 more than the largest level number of T.

Tree Terminology



- Binary trees T and T' are said to be **similar** if they have the same structure or, in other words, if they have the same shape. The trees are said to be **copies** if they are similar and if they have the same contents at corresponding nodes.

Consider the four binary trees in Fig. 7.2. The three trees (a), (c) and (d) are similar. In particular, the trees (a) and (c) are copies since they also have the same data at corresponding nodes. The tree (b) is neither similar nor a copy of the tree (d) because, in a binary tree, we distinguish between a left successor and a right successor even when there is only one successor.

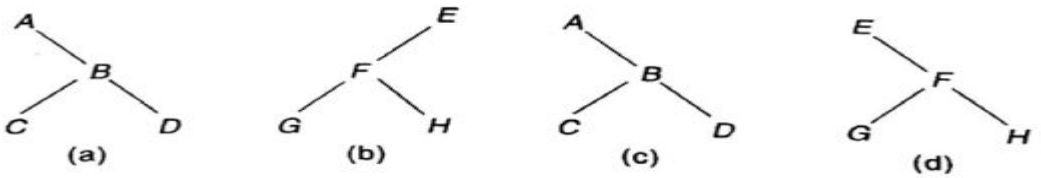
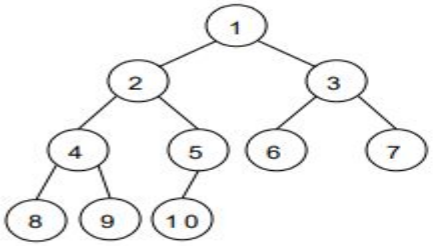


Fig. 7.2

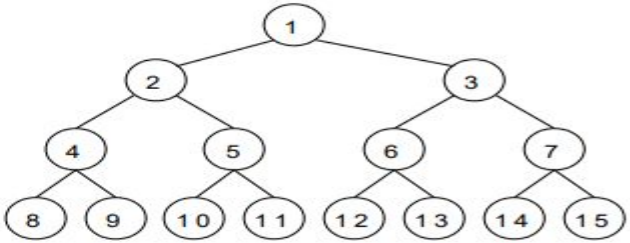
Tree Terminology

Full Binary tree

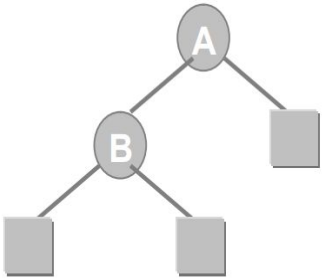
A full binary tree of height **h** has all its leaves at level **h-1**. Alternatively; All non leaf nodes of a full binary tree have two children, and the leaf nodes have no children. A full binary tree with height **h** has $\mathbf{2^h - 1}$ nodes.



Complete binary tree



Full binary tree



Extended 2-tree

Complete Binary Tree:

Consider a binary tree T. The tree T is said to be complete if all its levels, except possibly the last have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible.

Extended Binary Tree / 2-Tree:

A binary tree T is said to be a 2-tree or an extended binary tree if each node N has either 0 or 2 children. In such a case, the nodes, with 2 children are called **internal nodes**, and the node with 0 children are called **external node**.

Representing Binary Trees in Memory

Let T be a binary tree. Here we will discuss two ways of representing T in memory-

I. Linked Representation of Binary Tree
II. Sequential Representation of Binary Tree

Linked Representation of Binary Tree

Consider a binary tree T. Unless otherwise stated or implied, T will be maintained in memory by means of a *linked representation* which uses three parallel arrays, INFO, LEFT and RIGHT, and a pointer variable ROOT as follows. First of all, each node N of T will correspond to a location K such that:

(1) INFO[K] contains the data at the node N.
(2) LEFT[K] contains the location of the left child of node N.
(3) RIGHT[K] contains the location of the right child of node N.

Furthermore, ROOT will contain the location of the root R of T. If any subtree is empty, then the corresponding pointer will contain the null value; if the tree T itself is empty, then ROOT will contain the null value.

Representing Binary Trees in Memory

Example 7.3

Consider the binary tree T in Fig. 7.1. A schematic diagram of the linked representation of T appears in Fig. 7.6. Observe that each node is pictured with its three fields, and that the empty subtrees are pictured by using × for the null entries. Figure 7.7 shows how this linked representation may appear in memory. The choice of 20 elements for the arrays is arbitrary. Observe that the AVAIL list is maintained as a one-way list using the array LEFT.

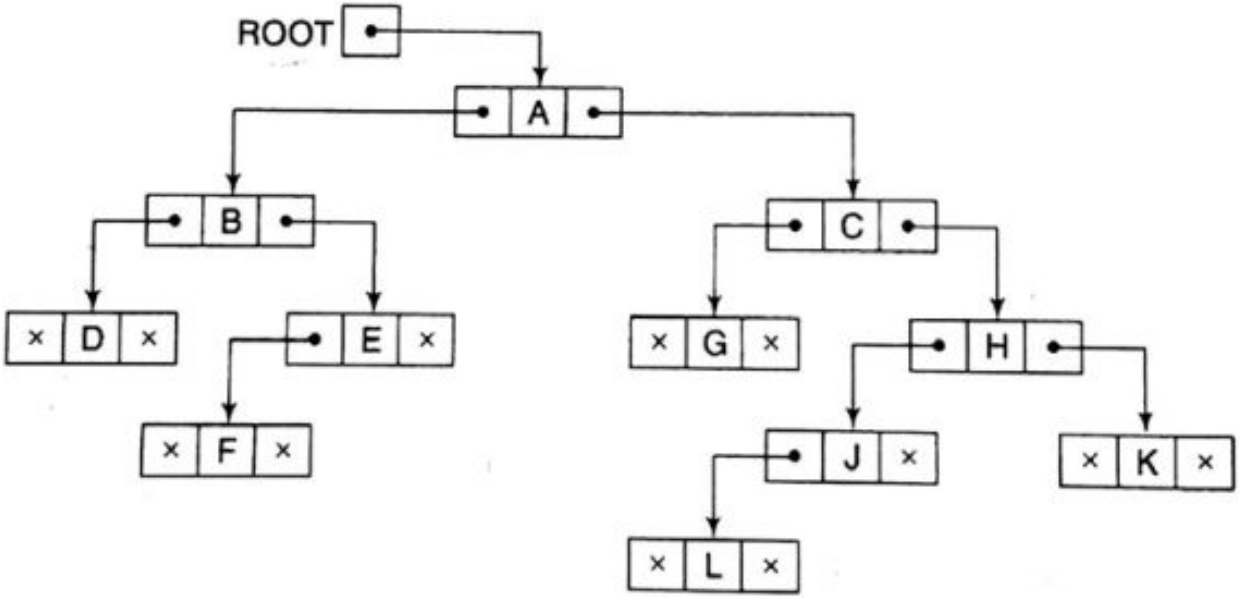


Fig. 7.6

Representing Binary Trees in Memory

Sequential Representation of Binary Tree

Suppose T is a binary tree that is complete or nearly complete. Then there is an efficient way of maintaining T in memory called the *sequential representation* of T. This representation uses only a single linear array TREE as follows:

(a) The root R of T is stored in TREE[1].

(b) If a node N occupies TREE[K], then its left child is stored in TREE[2 * K] and its right child is stored in TREE[2 * K + 1].

Again, NULL is used to indicate an empty subtree. In particular, TREE[1] = NULL indicates that the tree is empty.

The sequential representation of the binary tree T in Fig. 7.10(a) appears in Fig. 7.10(b).

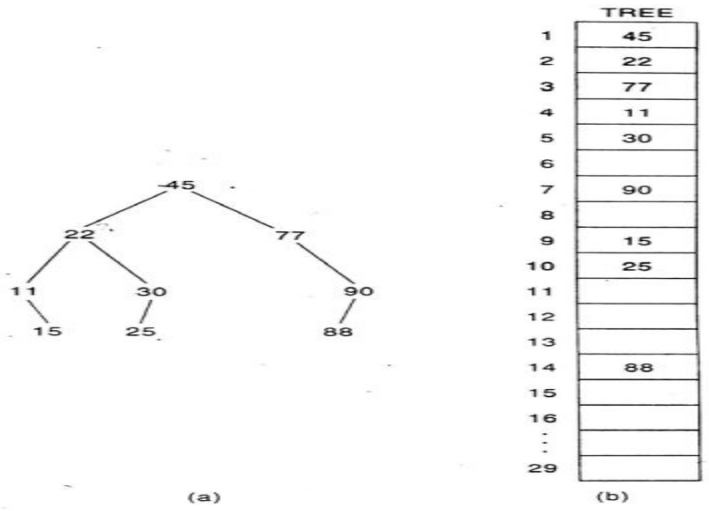


Fig. 7.10

Representing Binary Trees in Memory

- Array representation is good for complete binary tree, but it is wasteful for many other binary trees. The representation suffers from insertion and deletion of node from the middle of the tree, as it requires the moment of potentially many nodes to reflect the change in level number of this node. To overcome this difficulty we represent the binary tree in linked representation.
- The advantage of using linked representation of binary tree is that: Insertion and deletion involve no data movement and no movement of nodes except the rearrangement of pointers.
- The disadvantages of linked representation of binary tree includes: Given a node structure, it is difficult to determine its parent node. Memory spaces are wasted for storing NULL pointers for the nodes, which have no subtrees.

Traversing Binary Trees

A traversal of a tree T is a systematic way of accessing or visiting all the node of T. There are three standard ways of traversing a binary tree T with root R. these are :

1**. Preorder (N L R):**

a) Process the node/root.

b) Traverse the Left sub tree.

c) Traverse the Right sub tree.

2. **Inorder (L N R):**

a) Traverse the Left sub tree.

b) Process the node/root.

c) Traverse the Right sub tree.

3. **Postorder (L R N):**

a) Traverse the Left sub tree.

b) Traverse the Right sub tree.

c) Process the node/root.

Traversing Binary Trees

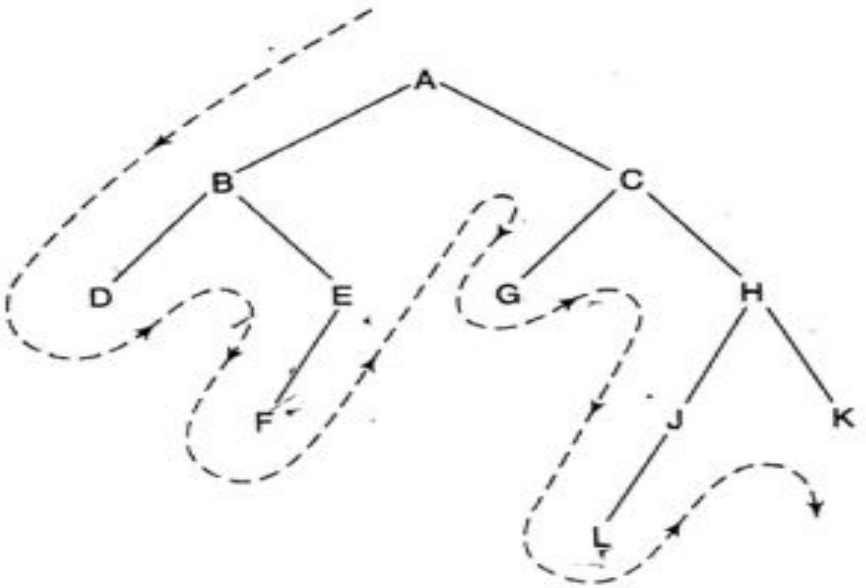
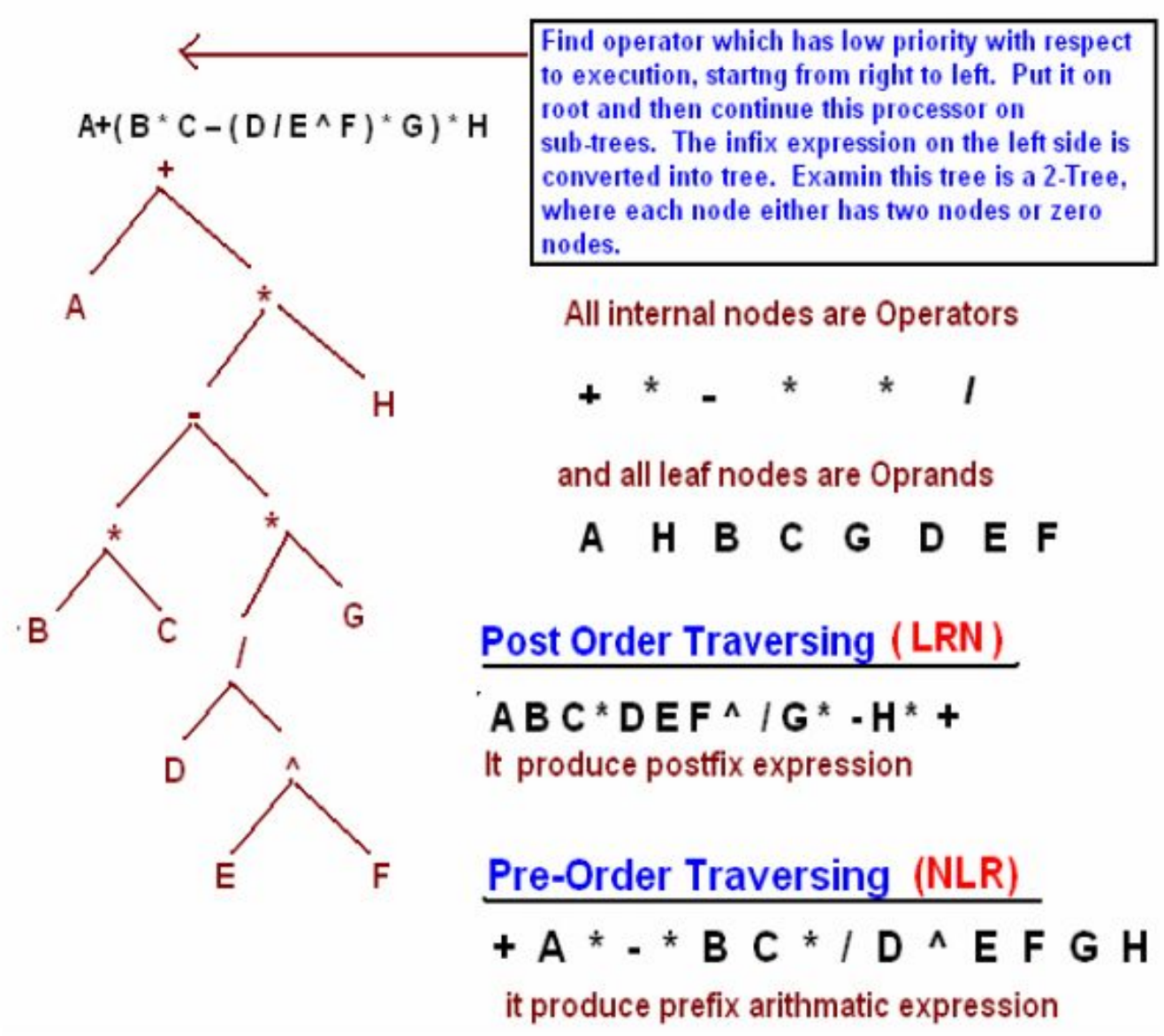


Fig. 7.12

The preorder traversal of T is ABDEFCGHJLK.

(Inorder)	D	B	F	E	A	G	C	L	J	H	K
(Postorder)	D	F	E	B	G	L	J	K	H	C	A

Preparing a Tree from an infix arithmetic expression



Formation of Binary Tree from its Traversal

- Sometimes it is required to construct a binary tree if its traversals are known. From a single traversal it is not possible to construct unique binary tree. However, if two traversals is given then corresponding tree can be drawn uniquely.
- The basic principle for formulation is as follows:

  If the preorder traversal is given, then the first node is the root node. If the postorder traversal is given then the last node is the root node. Once the root node is identified, all the nodes in the left sub-trees and right sub- trees of the root node can be identified. Same technique can be applied repeatedly to form sub- trees.
- It can be noted that, two traversal are essential out of which one should be **inorder traversal** and another preorder or postorder; alternatively, given preorder and postorder traversals, binary tree cannot be obtained uniquely.

Formation of Binary Tree from its Traversal

7.2 A binary tree T has 9 nodes. The inorder and preorder traversals of T yield the followi sequences of nodes:

Inorder:	E	A	C	K	F	H	D	B	G
Preorder:	F	A	E	K	C	D	H	G	B

Draw the tree T.

The tree T is drawn from its root downward as follows.

(a) The root of T is obtained by choosing the first node in its preorder. Thus F is the root of T.

(b) The left child of the node F is obtained as follows. First use the inorder of T to find the nodes in the left subtree T_1 of F. Thus T_1 consists of the nodes E, A, C and K. Then the left child of F is obtained by choosing the first node in the preorder of T_1 (which appears in the preorder of T). Thus A is the left son of F.

(c) Similarly, the right subtree T_2 of F consists of the nodes H, D, B and G, and D is the root of T_2, that is, D is the right child of F.

Repeating the above process with each new node, we finally obtain the required tree in Fig. 7.74.

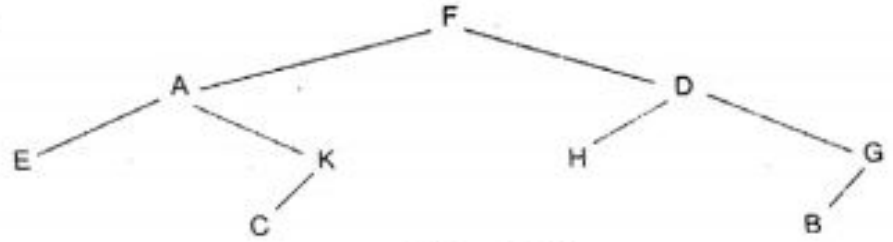


Fig. 7.74

Binary Search Tree

Suppose T is a binary tree. Then T is called a **binary search tree** or **binary sorted tree** if each node N of T has the following property:

The value of at N (node) is greater than every value in the left sub tree of N and is less than or equal to every value in the right sub tree of N.

Consider the binary tree T in Fig. 7.21. T is a binary search tree; that is, every node N in T exceeds every number in its left subtree and is less than every number in its right subtree. Suppose the 23 were replaced by 35. Then T would still be a binary search tree. On the other hand, suppose the 23 were replaced by 40. Then T would not be a binary search tree, since the 38 would not be greater than the 40 in its left subtree.

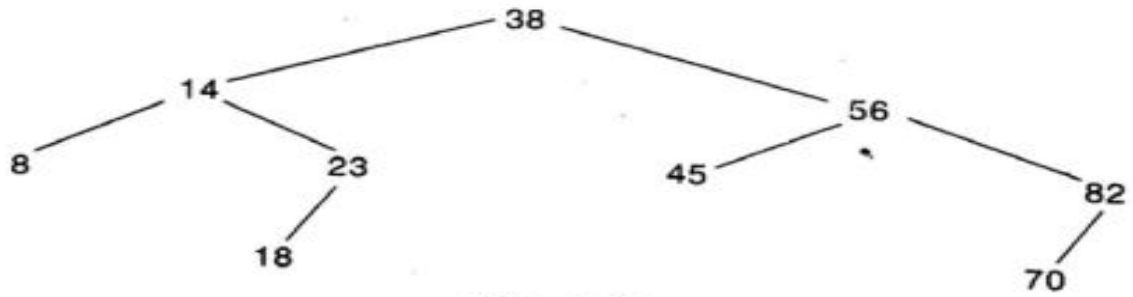


Fig. 7.21

Searching & Inserting in Binary Search Trees

Suppose an ITEM of information is given. The following algorithm finds the location of ITEM in the binary search tree T, or inserts ITEM as a new node in its appropriate place in the tree.

(a) Compare ITEM with the root node N of the tree.

(i) If ITEM < N, proceed to the left child of N.
(ii) If ITEM > N, proceed to the right child of N.

(b) Repeat Step (a) until one of the following occurs:

(i) We meet a node N such that ITEM = N. In this case the search is successful.
(ii) We meet an empty subtree, which indicates that the search is unsuccessful, and we insert ITEM in place of the empty subtree.

In other words, proceed from the root R down through the tree T until finding ITEM in T or inserting ITEM as a terminal node in T.

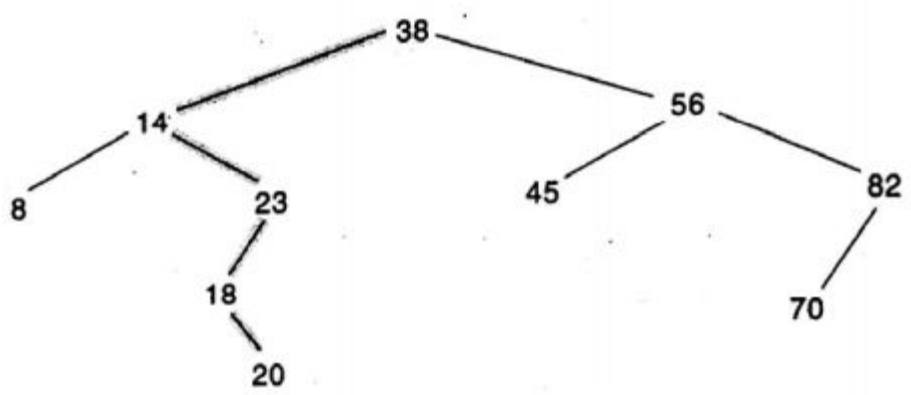


Fig. 7.22 *ITEM = 20 Inserted*

Binary Search Tree

Suppose the following six numbers are inserted in order into an empty binary search tree:

40, 60, 50, 33, 55, 11

Figure 7.23 shows the six stages of the tree. We emphasize that if the six numbers were given in a different order, then the tree might be different and we might have a different depth.

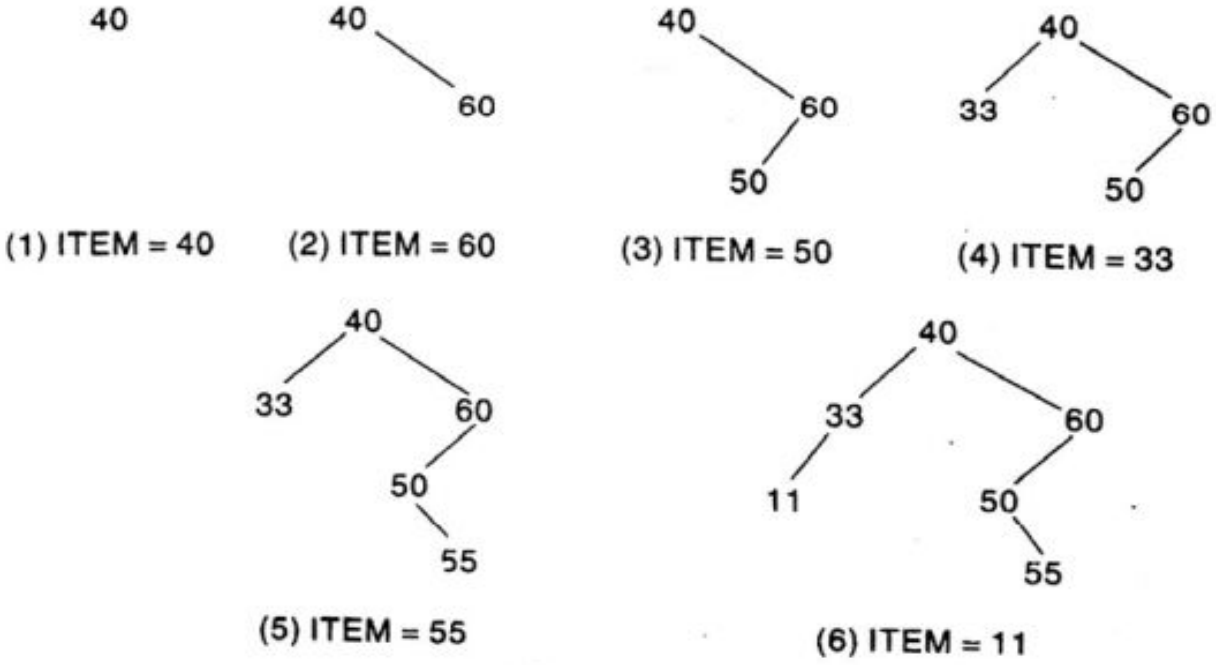


Fig. 7.23

Searching & Inserting in Binary Search Trees

FIND(INFO, LEFT, RIGHT, ROOT, ITEM, LOC, PAR)
A binary search tree T is in memory and an ITEM of information is given. This procedure finds the location LOC of ITEM in T and also the location PAR of the parent of ITEM. There are three special cases:

(i) LOC = NULL and PAR = NULL will indicate that the tree is empty.
(ii) LOC ≠ NULL and PAR = NULL will indicate that ITEM is the root of T.
(iii) LOC = NULL and PAR ≠ NULL will indicate that ITEM is not in T and can be added to T as a child of the node N with location PAR.

1. [Tree empty?]
   If ROOT = NULL, then: Set LOC := NULL and PAR := NULL, and Return.
2. [ITEM at root?]
   If ITEM = INFO[ROOT], then: Set LOC := ROOT and PAB := NULL, and Return.
3. [Initialize pointers PTR and SAVE.]
   If ITEM < INFO[ROOT], then:
   Set PTR := LEFT[ROOT] and SAVE := ROOT.
   Else:
   Set PTR := RIGHT[ROOT] and SAVE := ROOT.
   [End of If structure.]
4. Repeat Steps 5 and 6 while PTR ≠ NULL:
5. [ITEM found?]
   If ITEM = INFO[PTR], then: Set LOC := PTR and PAR := SAVE, and Return.
6. If ITEM < INFO[PTR], then:
   Set SAVE := PTR and PTR := LEFT[PTR].
   Else:
   Set SAVE := PTR and PTR := RIGHT[PTR].
   [End of If structure.]
   [End of Step 4 loop.]
7. [Search unsuccessful.] Set LOC := NULL and PAR := SAVE.
8. Exit.

Searching & Inserting in Binary Search Trees

INSBST(INFO, LEFT, RIGHT, ROOT, AVAIL, ITEM, LOC)
A binary search tree T is in memory and an ITEM of information is given. This algorithm finds the location LOC of ITEM in T or adds ITEM as a new node in T at location LOC.

1. Call FIND(INFO, LEFT, RIGHT, ROOT, ITEM, LOC, PAR).
   [Procedure 7.4.]
2. If LOC ≠ NULL, then Exit.
3. [Copy ITEM into new node in AVAIL list.]
   (a) If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
   (b) Set NEW := AVAIL, AVAIL := LEFT[AVAIL] and
       INFO[NEW] := ITEM.
   (c) Set LOC := NEW, LEFT[NEW] := NULL and
       RIGHT[NEW] := NULL.
4. [Add ITEM to tree.]
   If PAR = NULL, then:
       Set ROOT := NEW.
   Else if ITEM < INFO[PAR], then:
       Set LEFT[PAR] := NEW.
   Else:
       Set RIGHT[PAR] := NEW.
   [End of If structure.]
5. Exit.

Complexity of the Searching Algorithm

Suppose we are searching for an item of information in a binary search tree T. Observe that the number of comparisons is bounded by the depth of the tree. This comes from the fact that we proceed down a single path of the tree. Accordingly, the running time of the search will be proportional to the depth of the tree.

Suppose we are given n data items, A_1 A_2, ..., A_N, and suppose the items are inserted in order into a binary search tree T. Recall that there are $n!$ permutations of the n items (Sec. 2.2). Each such permutation will give rise to a corresponding tree. It can be shown that the average depth of the $n!$ trees is approximately $c \log_2 n$, where $c = 1.4$. Accordingly, the average running time $f(n)$ to search for an item in a binary tree T with n elements is proportional to $\log_2 n$, that is, $f(n) = O(\log_2 n)$.

Deleting in a Binary Search Tree

Suppose T is a binary search tree, and suppose an ITEM of information is given. This section gives an algorithm which deletes ITEM from the tree T.

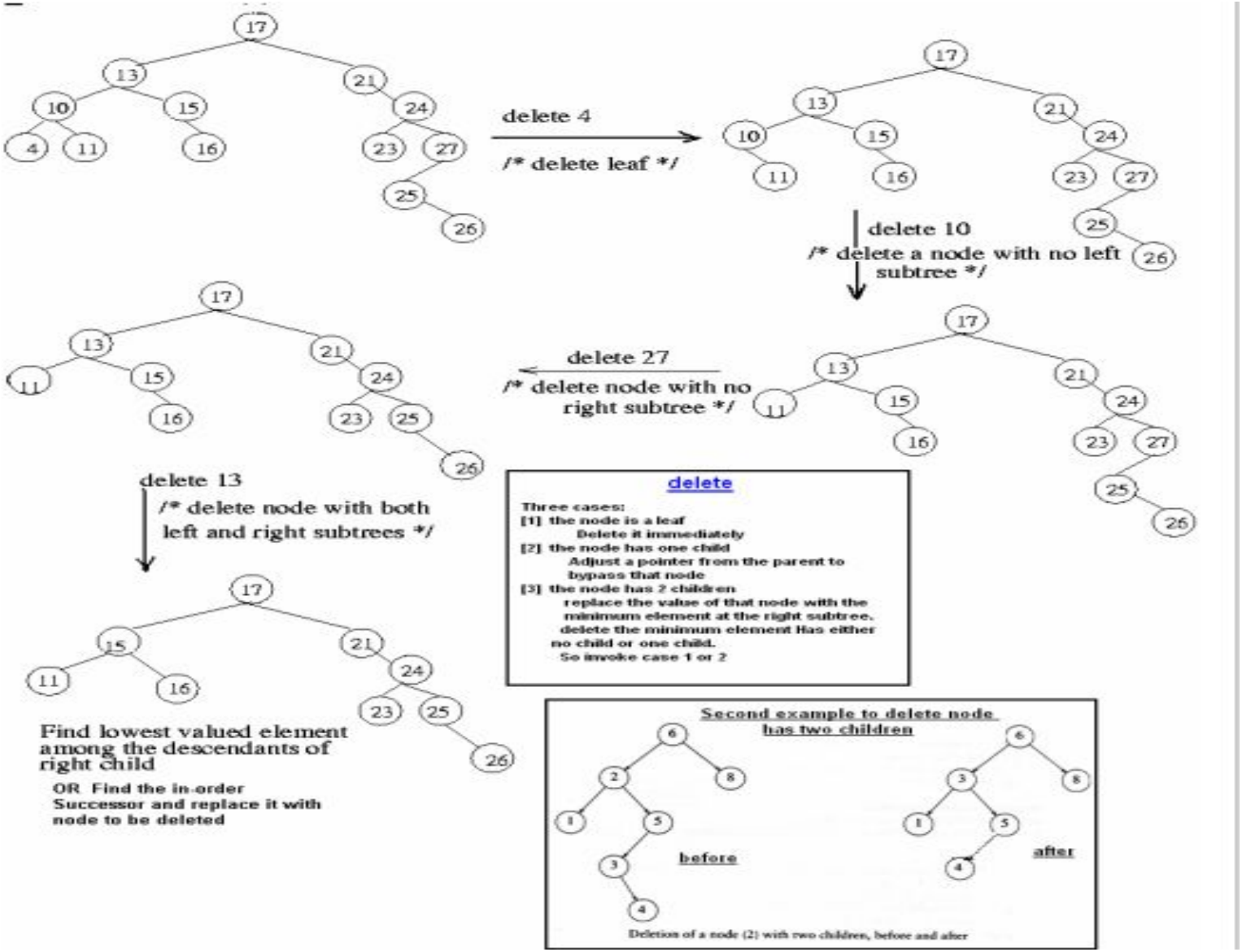
The deletion algorithm first uses Procedure 7.4 to find the location of the node N which contains ITEM and also the location of the parent node P(N). The way N is deleted from the tree depends primarily on the number of children of node N. There are three cases:

Case 1. N has no children. Then N is deleted from T by simply replacing the location of N in the parent node P(N) by the null pointer.

Case 2. N has exactly one child. Then N is deleted from T by simply replacing the location of N in P(N) by the location of the only child of N.

Case 3. N has two children. Let S(N) denote the inorder successor of N. (The reader can verify that S(N) does not have a left child.) Then N is deleted from T by first deleting S(N) from T (by using Case 1 or Case 2) and then replacing node N in T by the node S(N).

Deleting in a Binary Search Tree



Deleting in a Binary Search Tree

Procedure 7.6: CASEA(INFO, LEFT, RIGHT, ROOT, LOC, PAR)

This procedure deletes the node N at location LOC, where N does not have two children. The pointer PAR gives the location of the parent of N, or else PAR = NULL indicates that N is the root node. The pointer CHILD gives the location of the only child of N, or else CHILD = NULL indicates N has no children.

```
1. [Initializes CHILD.]
   If LEFT[LOC] = NULL and RIGHT[LOC] = NULL, then:
       Set CHILD := NULL.
   Else if LEFT[LOC] ≠ NULL, then:
       Set CHILD := LEFT[LOC].
   Else
       Set CHILD := RIGHT[LOC].
   [End of If structure.]
2. If PAR ≠ NULL, then:
       If LOC = LEFT[PAR], then:
           Set LEFT[PAR] := CHILD.
       Else:
           Set RIGHT[PAR] := CHILD.
       [End of If structure.]
   Else:
       Set ROOT := CHILD.
   [End of If structure.]
3. Return.
```

Deleting in a Binary Search Tree

Procedure 7.7: CASEB(INFO, LEFT, RIGHT, ROOT, LOC, PAR)
This procedure will delete the node N at location LOC, where N has two children. The pointer PAR gives the location of the parent of N, or else PAR = NULL indicates that N is the root node. The pointer SUC gives the location of the inorder successor of N, and PARSUC gives the location of the parent of the inorder successor.

1. [Find SUC and PARSUC.]
  (a) Set PTR := RIGHT[LOC] and SAVE := LOC.
  (b) Repeat while LEFT[PTR] ≠ NULL:
    Set SAVE := PTR and PTR := LEFT[PTR].
  [End of loop.]
  (c) Set SUC := PTR and PARSUC := SAVE.

2. [Delete inorder successor, using Procedure 7.6.]
Call CASEA(INFO, LEFT, RIGHT, ROOT, SUC, PARSUC).

3. [Replace node N by its inorder successor.]
  (a) If PAR ≠ NULL, then:
    If LOC = LEFT[PAR], then:
      Set LEFT[PAR] := SUC.
    Else:
      Set RIGHT[PAR] := SUC.
    [End of If structure.]
  Else:
    Set ROOT := SUC.
  [End of If structure.]
  (b) Set LEFT[SUC] := LEFT[LOC] and
  RIGHT[SUC] := RIGHT[LOC].

4. Return.

Deleting in a Binary Search Tree

Algorithm 7.8: DEL(INFO, LEFT, RIGHT, ROOT, AVAIL, ITEM)
A binary search tree T is in memory, and an ITEM of information is given. This algorithm deletes ITEM from the tree.

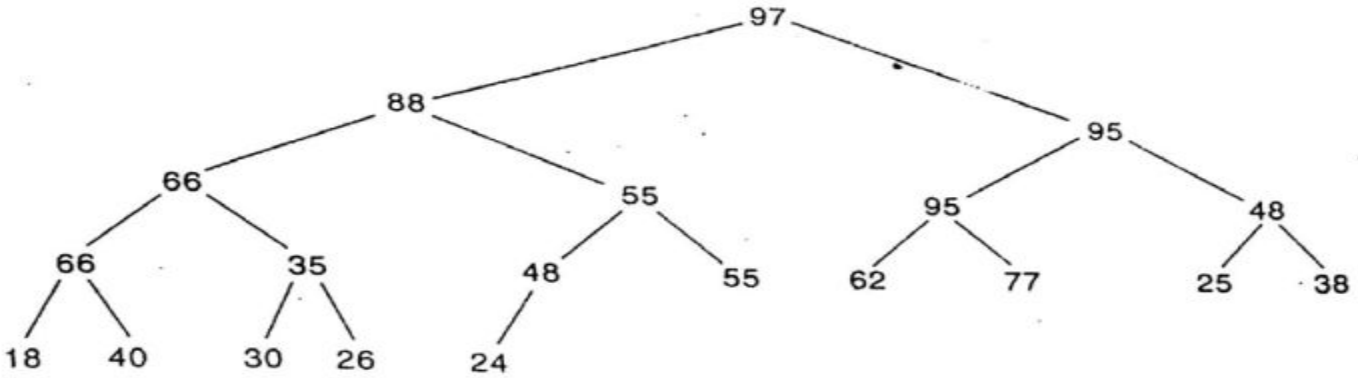
1. [Find the locations of ITEM and its parent, using Procedure 7.4.]
   Call FIND(INFO, LEFT, RIGHT, ROOT, ITEM, LOC, PAR).
2. [ITEM in tree?]
   If LOC = NULL, then: Write: ITEM not in tree, and Exit.
3. [Delete node containing ITEM.]
   If RIGHT[LOC] ≠ NULL and LEFT[LOC] ≠ NULL, then:
      Call CASEB(INFO, LEFT, RIGHT, ROOT, LOC, PAR).
   Else:
      Call CASEA(INFO, LEFT, RIGHT, ROOT, LOC, PAR).
   [End of If structure.]
4. [Return deleted node to the AVAIL list.]
   Set LEFT[LOC] := AVAIL and AVAIL := LOC.
5. Exit.

Lecture – 15

Tree 2

Heap

Suppose H is a complete binary tree with n elements. (Unless otherwise stated, we assume that H is maintained in memory by a linear array TREE using the sequential representation of H, not a linked representation.) Then H is called a *heap*, or a *maxheap*, if each node N of H has the following property: *The value at* N *is greater than or equal to the value at each* of *the children of* N. Accordingly, the value at N is greater than or equal to the value at any of the descendants of N. (A *minheap* is defined analogously: The value at N is less than or equal to the value at any of the children of N.)



(a) Heap

TREE

97	88	95	66	55	95	48	66	35	48	55	62	77	25	38	18	40	30	26	24
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

(b) Sequential representation

Fig. 7.58

Inserting into a Heap

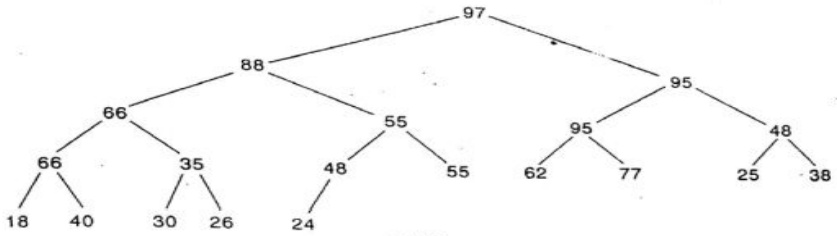
Suppose H is a heap with N elements, and suppose an ITEM of information is given. We insert ITEM into the heap H as follows:

(1) First adjoin ITEM at the end of H so that H is still a complete tree, but not necessarily a heap.
(2) Then let ITEM rise to its "appropriate place" in H so that H is finally a heap.

We illustrate the way this procedure works before stating the procedure formally.

Example 7.33

Consider the heap H in Fig. 7.58. Suppose we want to add ITEM = 70 to H. First we adjoin 70 as the next element in the complete tree; that is, we set TREE[21] = 70. Then 70 is the right child of TREE[10] = 48. The path from 70 to the root of H is pictured in Fig. 7.59(a). We now find the appropriate place of 70 in the heap as follows:



(a) Heap

TREE

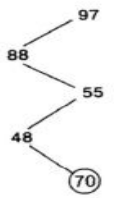
97	88	95	66	55	95	48	66	35	48	55	62	77	25	38	18	40	30	26	24
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

(b) Sequential representation

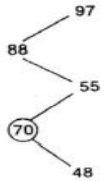
Fig. 7.58

(a) Compare 70 with its parent, 48. Since 70 is greater than 48, interchange 70 and 48; the path will now look like Fig. 7.59(b).

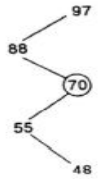
(b) Compare 70 with its new parent, 55. Since 70 is greater than 55, interchange 70 and 55; the path will now look like Fig. 7.59(c).



(a)



(b)



(c)

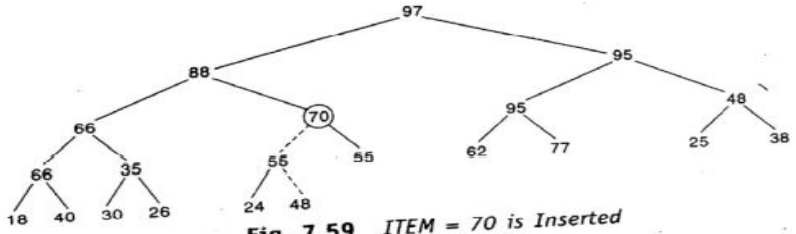


Fig. 7.59 *ITEM = 70 is Inserted*

Remark: One must verify that the above procedure does always yield a heap as a final tree, that is, that nothing else has been disturbed. This is easy to see, and we leave this verification to the reader.

Example 7.34

Suppose we want to build a heap H from the following list of numbers:

44, 30, 50, 22, 60, 55, 77, 55

This can be accomplished by inserting the eight numbers one after the other into an empty heap H using the above procedure. Figure 7.60(a) through (h) shows the respective pictures of the heap after each of the eight elements has been inserted. Again, the dotted line indicates that an exchange has taken place during the insertion of the given ITEM of information.

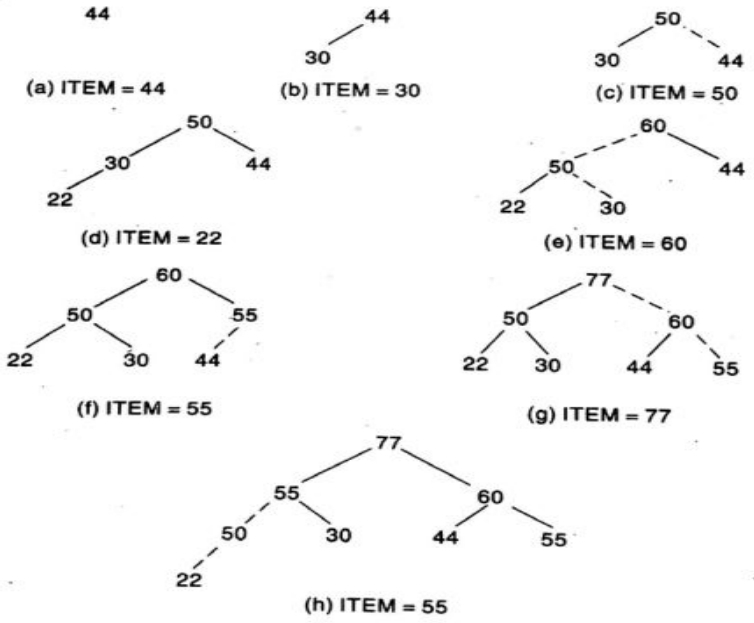


Fig. 7.60 *Building a Heap*

The formal statement of our insertion procedure follows:

Procedure 7.9: INSHEAP(TREE, N, ITEM)

A heap H with N elements is stored in the array TREE, and an ITEM of information is given. This procedure inserts ITEM as a new element of H. PTR gives the location of ITEM as it rises in the tree, and PAR denotes the location of the parent of ITEM.

1. [Add new node to H and initialize PTR.]
Set N := N + 1 and PTR := N.
2. [Find location to insert ITEM.]
Repeat Steps 3 to 6 while PTR < 1.
3. Set PAR := ⌊PTR/2⌋. [Location of parent node.]
4. If ITEM ≤ TREE[PAR], then:
Set TREE[PTR] := ITEM, and Return.
[End of If structure.]
5. Set TREE[PTR] := TREE[PAR]. [Moves node down.]
6. Set PTR := PAR. [Updates PTR.]
[End of Step 2 loop.]
7. [Assign ITEM as the root of H.]
Set TREE[1] := ITEM.
8. Return.

Observe that ITEM is not assigned to an element of the array TREE until the appropriate place for ITEM is found. Step 7 takes care of the special case that ITEM rises to the root TREE[1].

Suppose an array A with N elements is given. By repeatedly applying Procedure 7.9 to A, that is, by executing

Call INSHEAP(A, J, A[J + 1])

for J = 1, 2, ..., N − 1, we can build a heap H out of the array A.

Deleting the Root of a Heap

Suppose H is a heap with N elements, and suppose we want to delete the root R of H. This is accomplished as follows:

(1) Assign the root R to some variable ITEM.
(2) Replace the deleted node R by the last node L of H so that H is still a complete tree, but not necessarily a heap.
(3) (Reheap) Let L sink to its appropriate place in H so that H is finally a heap.

Again we illustrate the way the procedure works before stating the procedure formally.

Example 7.35

Consider the heap H in Fig. 7.61(a), where R = 95 is the root and L = 22 is the last node of the tree. Step 1 of the above procedure deletes R = 95, and Step 2 replaces R = 95 by L = 22. This gives the complete tree in Fig. 7.61(b), which is not a heap. Observe, however, that both the right and left subtrees of 22 are still heaps. Applying Step 3, we find the appropriate place of 22 in the heap as follows:

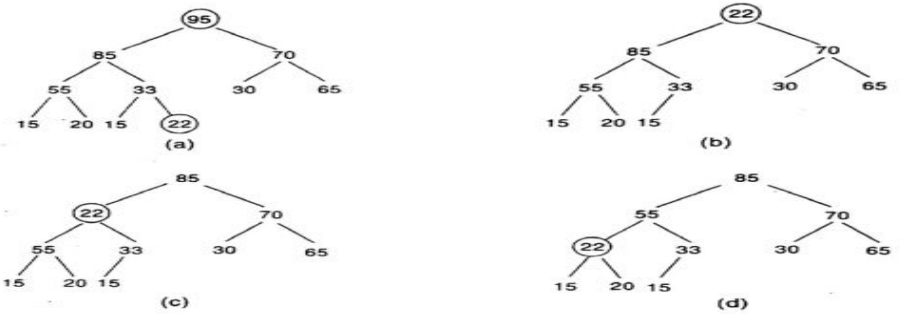


Fig. 7.61 *Reheaping*

(a) Compare 22 with its two children, 85 and 70. Since 22 is less than the larger child, 85, interchange 22 and 85 so the tree now looks like Fig. 7.61(c).
(b) Compare 22 with its two new children, 55 and 33. Since 22 is less than the larger child, 55, interchange 22 and 55 so the tree now looks like Fig. 7.61(d).
(c) Compare 22 with its new children, 15 and 20. Since 22 is greater than both children, node 22 has dropped to its appropriate place in H.

Thus Fig. 7.61(d) is the required heap H without its original root R.

Remark: As with inserting an element into a heap, one must verify that the above procedure does always yield a heap as a final tree. Again we leave this verification to the reader. We also note that Step 3 of the procedure may not end until the node L reaches the bottom of the tree, i.e., until L has no children.

The formal statement of our procedure follows.

Procedure 7.10: DELHEAP(TREE, N, ITEM)

A heap H with N elements is stored in the array TREE. This procedure assigns the root TREE[1] of H to the variable ITEM and then reheaps the remaining elements. The variable LAST saves the value of the original last node of H. The pointers PTR, LEFT and RIGHT give the locations of LAST and its left and right children as LAST sinks in the tree.

1. Set ITEM := TREE[1]. [Removes root of H.]
2. Set LAST := TREE[N] and N := N – 1. [Removes last node of H.]
3. Set PTR := 1, LEFT := 2 and RIGHT := 3. [Initializes pointers.]
4. Repeat Steps 5 to 7 while RIGHT ≤ N:
5. If LAST ≥ TREE[LEFT] and LAST ≥ TREE[RIGHT], then:
Set TREE[PTR] := LAST and Return.
[End of If structure.]
6. IF TREE[RIGHT] ≤ TREE[LEFT], then:
Set TREE[PTR] := TREE[LEFT] and PTR := LEFT.
Else:
Set TREE[PTR] := TREE[RIGHT] and PTR := RIGHT.
[End of If structure.]
7. Set LEFT := 2*PTR and RIGHT := LEFT + 1.
[End of Step 4 loop.]
8. If LEFT = N and if LAST < TREE[LEFT], then: Set PTR := LEFT.
9. Set TREE[PTR] := LAST.
10. Return.

The Step 4 loop repeats as long as LAST has a right child. Step 8 takes care of the special case in which LAST does not have a right child but does have a left child (which has to be the last node in H). The reason for the two "If" statements in Step 8 is that TREE[LEFT] may not be defined when LEFT > N.

Application to Sorting

Suppose an array A with N elements is given. The heapsort algorithm to sort A consists of the two following phases:

Phase A: Build a heap H out of the elements of A.
Phase B: Repeatedly delete the root element of H.

Since the root of H always contains the largest node in H, Phase B deletes the elements of A in decreasing order. A formal statement of the algorithm, which uses Procedures 7.9 and 7.10, follows.

Algorithm 7.11: HEAPSORT(A, N)
An array A with N elements is given. This algorithm sorts the elements of A.

1. [Build a heap H, using Procedure 7.9.]
   Repeat for J = 1 to N − 1:
   Call INSHEAP(A, J, A[J + 1]).
   [End of loop.]
2. [Sort A by repeatedly deleting the root of H, using Procedure 7.10.]
   Repeat while N > 1:
   (a) Call DELHEAP(A, N, ITEM).
   (b) Set A[N + 1] := ITEM.
   [End of Loop.]
3. Exit.

The purpose of Step 2(b) is to save space. That is, one could use another array B to hold the sorted elements of A and replace Step 2(b) by

Set B[N + 1] := ITEM

However, the reader can verify that the given Step 2(b) does not interfere with the algorithm, since A[N + 1] does not belong to the heap H.

Complexity of Heapsort

Suppose the heapsort algorithm is applied to an array A with n elements. The algorithm has two phases, and we analyze the complexity of each phase separately.

Phase A. Suppose H is a heap. Observe that the number of comparisons to find the appropriate place of a new element ITEM in H cannot exceed the depth of H. Since H is a complete tree, its depth is bounded by $\log_2 m$ where m is the number of elements in H. Accordingly, the total number $g(n)$ of comparisons to insert the n elements of A into H is bounded as follows:

$$g(n) \leq n \log_2 n$$

Consequently, the running time of Phase A of heapsort is proportional to $n \log_2 n$.

Phase B. Suppose H is a complete tree with m elements, and suppose the left and right subtrees of H are heaps and L is the root of H. Observe that reheaping uses 4 comparisons to move the node L one step down the tree H. Since the depth of H does not exceed $\log_2 m$, reheaping uses at most 4 $\log_2 m$ comparisons to find the appropriate place of L in the tree H. This means that the total number $h(n)$ of comparisons to delete the n elements of A from H, which requires reheaping n times, is bounded as follows:

$$h(n) \leq 4n \log_2 n$$

Accordingly, the running time of Phase B of heapsort is also proportional to $n \log_2 n$.

Since each phase requires time proportional to $n \log_2 n$, the running time to sort the n-element array A using heapsort is proportional to $n \log_2 n$, that is, $f(n) = O(n \log_2 n)$. Observe that this gives a worst-case complexity of the heapsort algorithm. This contrasts with the following two sorting algorithms already studied:

(1) *Bubble sort* (Sec. 4.6). The running time of bubble sort is $O(n^2)$.
(2) *Quicksort* (Sec. 6.5). The average running time of quicksort is $O(n \log_2 n)$, the same as heapsort, but the worst-case running time of quicksort is $O(n^2)$, the same as bubble sort.

Other sorting algorithms are investigated in Chapter 9.

7.19 GENERAL TREES

A *general tree* (sometimes called a *tree*) is defined to be a nonempty finite set T of elements, called *nodes*, such that:

(1) T contains a distinguished element R, called the *root* of T.
(2) The remaining elements of T form an ordered collection of zero or more disjoint trees $T_1, T_2, \ldots, T_m$.

The trees $T_1, T_2, \ldots, T_m$ are called *subtrees* of the root R, and the roots of $T_1, T_2, \ldots, T_m$ are called *successors* of R.

Terminology from family relationships, graph theory and horticulture is used for general trees in the same way as for binary trees. In particular, if N is a node with successors $S_1, S_2, \ldots, S_m$, then N is called the *parent* of the S_i's, the S_i's are called *children* of N, and the S_i's are called *siblings* of each other.

The term "tree" comes up, with slightly different meanings, in many different areas of mathematics and computer science. Here we assume that our general tree T is *rooted*, that is, that T has a distinguished node R called the root of T; and that T is *ordered*, that is, that the children of each node N of T have a specific order. These two properties are not always required for the definition of a tree.

Example 7.38

Figure 7.68 pictures a general tree T with 13 nodes,

$$A, B, C, D, E, F, G, H, J, K, L, M, N$$

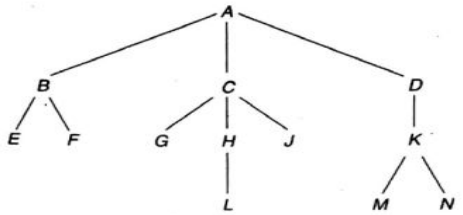


Fig. 7.68

Unless otherwise stated, the root of a tree T is the node at the top of the diagram, and the children of a node are ordered from left to right. Accordingly, A is the root of T, and A has three children; the first child B, the second child C and the third child D. Observe that:

(a) The node C has three children.
(b) Each of the nodes B and K has two children.
(c) Each of the nodes D and H has only one child.
(d) The nodes E, F, G, L, J, M and N have no children.

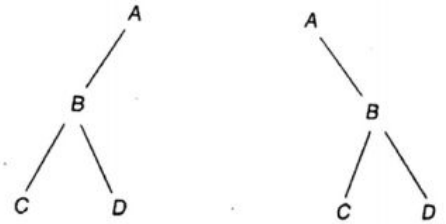
The last group of nodes, those with no children, are called *terminal nodes*.

A binary tree T' is not a special case of a general tree T: binary trees and general trees are different objects. The two basic differences follow:

(1) A binary tree T' may be empty, but a general tree T is nonempty.
(2) Suppose a node N has only one child. Then the child is distinguished as a left child or right child in a binary tree T', but no such distinction exists in a general tree T.

The second difference is illustrated by the trees T_1 and T_2 in Fig. 7.69. Specifically, as binary trees, T_1 and T_2 are distinct trees, since B is the left child of A in the tree T_1 but B is the right child of A in the tree T_2. On the other hand, there is no difference between the trees T_1 and T_2 as general trees.

A *forest* F is defined to be an ordered collection of zero or more distinct trees. Clearly, if we delete the root R from a general tree T, then we obtain the forest F consisting of the subtrees of R (which may be empty). Conversely, if F is a forest, then we may adjoin a node R to F to form a general tree T where R is the root of T and the subtrees of R consist of the original trees in F.



(a) Tree T_1 (b) Tree T_2

Fig. 7.69

Computer Representation of General Trees

Suppose T is a general tree. Unless otherwise stated or implied, T will be maintained in memory by means of a linked representation which uses three parallel arrays INFO, CHILD (or DOWN) and SIBL (or HORZ), and a pointer variable ROOT as follows. First of all, each node N of T will correspond to a location K such that:

(1) INFO[K] contains the data at node N.
(2) CHILD[K] contains the location of the first child of N. The condition CHILD[K] = NULL indicates that N has no children.
(3) SIBL[K] contains the location of the next sibling of N. The condition SIBL[K] = NULL indicates that N is the last child of its parent.

Furthermore, ROOT will contain the location of the root R of T. Although this representation may seem artificial, it has the important advantage that each node N of T, regardless of the number of children of N, will contain exactly three fields.

The above representation may easily be extended to represent a forest F consisting of trees T_1, T_2, ..., T_m by assuming the roots of the trees are siblings. In such a case, ROOT will contain the location of the root R_1 of the first tree T_1; or when F is empty, ROOT will equal NULL.

Example 7.39

Consider the general tree T in Fig. 7.68. Suppose the data of the nodes of T are stored in an array INFO as in Fig. 7.70(a). The structural relationships of T are obtained by assigning values to the pointer ROOT and the arrays CHILD and SIBL as follows:

(a) Since the root A of T is stored in INFO[2], set ROOT := 2.
(b) Since the first child of A is the node B, which is stored in INFO[3], set CHILD[2] := 3. Since A has no sibling, set SIBL[2] := NULL.
(c) Since the first child of B is the node E, which is stored in INFO[15], set CHILD[3] := 15. Since node C is the next sibling of B and C is stored in INFO[4], set SIBL[3] := 4.

	INFO
1	
2	A
3	B
4	C
5	
6	G
7	H
8	J
9	N
10	M
11	L
12	K
13	
14	F
15	E
16	D

(a)

	CHILD	SIBL
1	5	
2	3	0
3	15	4
4	6	16
5	13	
6	0	7
7	11	8
8	0	0
9	0	0
10	0	9
11	0	0
12	10	0
13	0	
14	0	0
15	0	14
16	12	0

ROOT = 2, AVAIL = 13

(b)

Fig. 7.70

And so on. Figure 7.70(b) gives the final values in CHILD and SIBL. Observe that the AVAIL list of empty nodes is maintained by the first array, CHILD, where AVAIL = 1.

Correspondence between General Trees and Binary Trees

Suppose T is a general tree. Then we may assign a unique binary tree T' to T as follows. First of all, the nodes of the binary tree T' will be the same as the nodes of the general tree T, and the root of T' will be the root of T. Let N be an arbitrary node of the binary tree T'. Then the left child of N in T' will be the first child of the node N in the general tree T and the right child of N in T' will be the next sibling of N in the general tree T.

Example 7.40

Consider the general tree T in Fig. 7.68. The reader can verify that the binary tree T' in Fig. 7.71 corresponds to the general tree T. Observe that by rotating counterclockwise the picture of T' in Fig. 7.71 until the edges pointing to right children are horizontal, we obtain a picture in which the nodes occupy the same relative position as the nodes in Fig. 7.68.

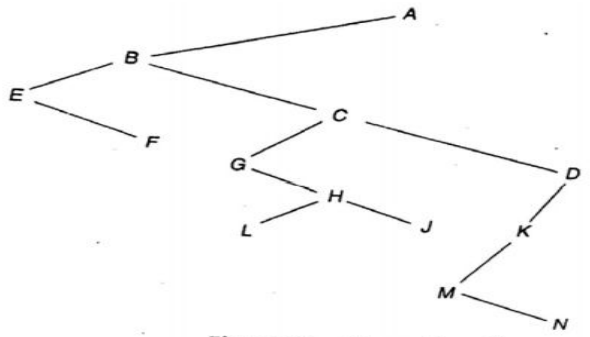


Fig. 7.71 *Binary Tree T'*

The computer representation of the general tree T and the linked representation of the corresponding binary tree T' are exactly the same except that the names of the arrays CHILD and SIBL for the general tree T will correspond to the names of the arrays LEFT and RIGHT for the binary tree T'. The importance of this correspondence is that certain algorithms that applied to binary trees, such as the traversal algorithms, may now apply to general trees.

Lecture – 16

Graphs

Graph

A graph is an abstract data structure that is used to implement the mathematical concept of graphs. It is basically a collection of vertices (also called nodes) and edges that connect these vertices.

Application of Graphs

Graphs are constructed for various types of applications such as:

- In circuit networks where points of connection are drawn as vertices and component wires become the edges of the graph.
- In transport networks where stations are drawn as vertices and routes become the edges of the graph.
- In maps that draw cities/states/regions as vertices and adjacency relations as edges.
- In program flow analysis where procedures or modules are treated as vertices and calls to these procedures are drawn as edges of the graph

Definition

A graph G is defined as an ordered set (V, E), where V(G) represents the set of vertices and E(G) represents the edges that connect these vertices.

Graph

Figure 13.1 shows a graph with V(G) = {A, B, C, D and E} and E(G) = {(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)}. Note that there are five vertices or nodes and six edges in the graph

A graph can be **directed** or **undirected**. In an undirected graph, edges do not have any direction associated with them. That is, if an edge is drawn between nodes A and B, then the nodes can be traversed from A to B as well as from B to A. Figure 13.1 shows an undirected graph because it does not give any information about the direction of the edges.

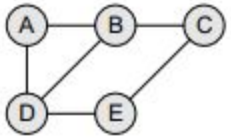


Figure 13.1 Undirected graph

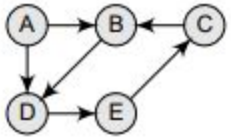


Figure 13.2 Directed graph

Look at Fig. 13.2 which shows a directed graph. In a directed graph, edges form an ordered pair. If there is an edge from A to B, then there is a path from A to B but not from B to A. The edge (A, B) is said to initiate from node A (also known as initial node) and terminate at node B (terminal node)

Graph Terminology

Adjacent nodes or neighbors:

For every edge, e = [u, v] that connects nodes u and v, the nodes u and v are the **end-points** and are said to be the **adjacent nodes** or **neighbors**.

Degree of a node:

Degree of a node u, **deg(u)**, is the total number of edges containing the node u. If **deg(u) = 0**, it means that u does not belong to any edge and such a node is known as an **isolated node**.

Path:

A path P written as P = {v_0 , v_1 , v_2 , ..., v_n), of length **n** from a node u to v is defined as a sequence of (**n+1**) nodes. Here, u = v_0 , v = v_n and v_{i-1} is adjacent to v_i for i = 1, 2, 3, ..., n.

Closed path

A path P is known as a closed path if the edge has the same end-points. That is, if $\mathbf{v_0 = v_n}$.

Simple path

A path P is known as a simple path if all the nodes in the path are distinct with an exception that v_0 may be equal to v_n . If $v_0 = v_n$, then the path is called a **closed simple path**.

Graph Terminology

Cycle:

A cycle is a closed simple path with length 3 or more. A **simple cycle** has no repeated edges or vertices (except the first and last vertices). A cycle of length k is called a **K-cycle**.

Connected graph

A graph is said to be connected if for any two vertices (u, v) in V there is a path from u to v. In a connected graph, there are no isolated nodes.

Proposition: *A graph G is connected if and only if there is a simple path between any two nodes in G*.

Complete graph

A graph G is said to be complete if every node u in G is adjacent to every other node v in G. A complete graph has **n(n–1)/2** edges, where **n** is the number of nodes in G.

Tree

A connected graph that does not have any cycle is called a tree. Therefore, a tree is treated as a special graph. If T is a finite tree with **m nodes**, then T will have **m-1 edges**. Figure 8.1(c) shows a tree.

Graph Terminology

Labelled graph or weighted graph

A graph is said to be labelled if every edge in the graph is assigned some data. In a **weighted graph**, the edges of the graph are assigned some weight or length. The weight of an edge denoted by w(e) is a positive value which indicates the cost of traversing the edge. Figure 8.1(d) shows a weighted graph.

Multiple edges

Distinct edges which connect the same end-points are called multiple edges. That is, e = [u, v] and e' = [u, v] are known as multiple edges of G.

Loop

An edge that has identical end-points is called a loop. That is, e = [u, u].

Multi-graph

A graph with multiple edges and/or loops is called a multi-graph. Figure 8.1(b) shows a multi-graph.

Size of a graph

The size of a graph is the total number of edges in it

Example 8.1

(a) Figure 8.1(a) is a picture of a connected graph with 5 nodes—A, B, C, D and E—and 7 edges:

$$[A, B],\quad [B, C],\quad [C, D],\quad [D, E],\quad [A, E],\quad [C, E]\quad [A, C]$$

There are two simple paths of length 2 from B to E: (B, A, E) and (B, C, E). There is only one simple path of length 2 from B to D: (B, C, D). We note that (B, A, D) is not a path, since $[A, D)$ is not an edge. There are two 4-cycles in the graph:

$$[A, B, C, E, A] \quad \text{and} \quad [A, C, D, E, A].$$

Note that $\deg(A) = 3$, since A belongs to 3 edges. Similarly, $\deg(C) = 4$ and $\deg(D) = 2$.

(b) Figure 8.1(b) is not a graph but a multigraph. The reason is that it has multiple edges—$e_4 = [B, C]$ and $e_5 = [B, C]$—and it has a loop, $e_6 = [D, D)$. The definition of a graph usually does not allow either multiple edges or loops.

(c) Figure 8.1(c) is a tree graph with $m = 6$ nodes and, consequently, $m - 1 = 5$ edges. The reader can verify that there is a unique simple path between any two nodes of the tree graph.

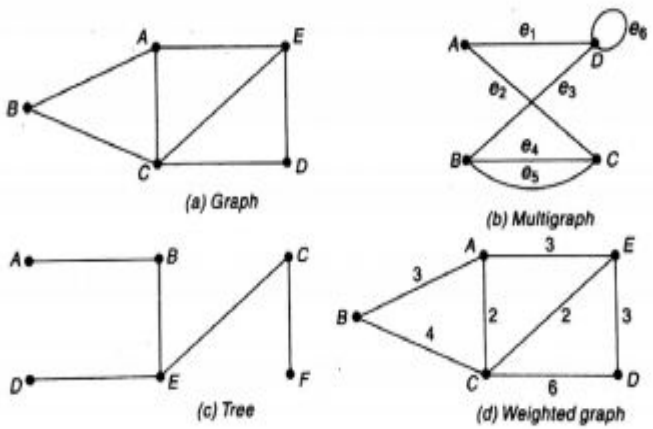


Fig. 8.1

(d) Figure 8.1(d) is the same graph as in Fig. 8.1(a), except that now the graph is weighted. Observe that $P_1 = (B, C, D)$ and $P_2 = (B, A, E, D)$ are both paths from node B to node D. Although P_2 contains more edges than P_1 the weight $w(P_2) = 9$ is less than the weight $w(P_1) = 10$.

Terminology of a Directed Graph

A directed graph G, also known as a **digraph,** is a graph in which every edge has a direction assigned to it. An edge of a directed graph is given as an ordered pair (u, v) of nodes in G.

For an edge e = (u, v) [e also called an **arc**] -

- The edge begins at u and terminates at v.
- u is known as the **origin** or **initial point** of e. Correspondingly, v is known as the **destination** or **terminal point** of e.
- u is the **predecessor** of v. Correspondingly, v is the **successor** or **neighbor** of u.
- u is adjacent to v, and v is adjacent from u.

Out-degree of a node

The out-degree of a node u, written as **outdeg(u)**, is the number of edges that originate at u.

In-degree of a node

The in-degree of a node u, written as **indeg(u)**, is the number of edges that terminate at u.

Pendant vertex (also known as leaf vertex) - A vertex with degree one.

Terminology of a Directed Graph

Source

A node u is known as a source if it has a positive out-degree but a zero in-degree.

Sink

A node u is known as a sink if it has a positive in-degree but a zero out-degree.

Reachability

A node v is said to be **reachable** from node u, if and only if there exists a (directed) path from node u to node v.

Strongly connected directed graph

A digraph is said to be strongly connected if and only if there exists a path between every pair of nodes in G. That is, if there is a path from node u to v, then there must be a path from node v to u.

Unilaterally connected graph

A digraph is said to be unilaterally connected if there exists a path between any pair of nodes u, v in G such that there is a path from u to v or a path from v to u, but not both.

Terminology of a Directed Graph

Weakly connected digraph

A directed graph is said to be weakly connected if it is connected by ignoring the direction of edges. That is, in such a graph, it is possible to reach any node from any other node by traversing edges in any direction (may not be in the direction they point). The nodes in a weakly connected directed graph must have either out-degree or in-degree of at least 1.

Parallel/Multiple edges

Distinct edges which connect the same end-points are called multiple edges. That is, e = (u, v) and e' = (u, v) are known as multiple edges of G.

Simple directed graph

A directed graph G is said to be a simple directed graph if and only if it has no parallel edges. However, a simple directed graph may contain cycles with an exception that it cannot have more than one loop at a given node.

Example 8.2

Figure 8.2 shows a directed graph G with 4 nodes and 7 (directed) edges. The edges e_2 and e_3 are said to be *parallel*, since each begins at B and ends at A. The edge e_7 is a *loop*, since it begins and ends at the same point, B. The sequence $P_1 = (D, C, B, A)$ is not a path, since (C, B) is not an edge—that is, the direction of the edge $e_5 = (B, C)$ does not agree with the direction of the path P_1. On the other hand, $P_2 = (D, B, A)$ is a path from D to A, since (D, B) and (B, A) are edges. Thus A is reachable from D. There is no path from C to any other node, so G is not strongly connected. However, G is unilaterally connected. Note that indeg$(D) = 1$ and outdeg$(D) = 2$. Node C is a sink, since indeg$(C) = 2$ but outdeg$(C) = 0$. No node in G is a source.

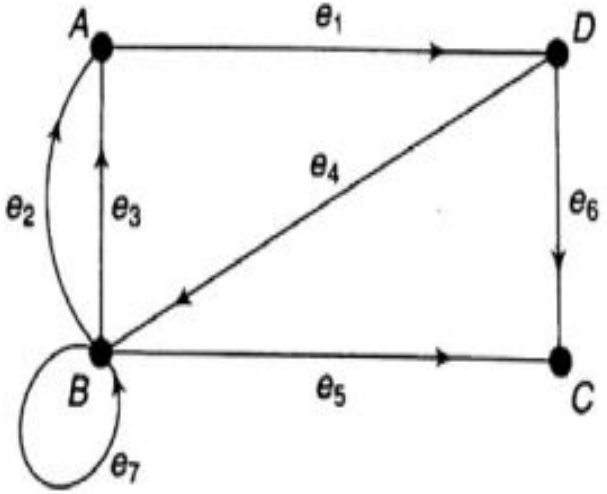


Fig. 8.2

Representation of Graphs

There are two common ways of storing graphs in the computer's memory. They are:

1. Sequential representation by using an adjacency matrix
2. Linked representation by using an adjacency list

Adjacency matrix

Suppose G is a simple directed graph with m nodes, and suppose the nodes of G have been ordered and are called $v_1, v_2, \ldots\ldots, v_m$. Then the adjacency matrix $A = (a_{ij})$ of the graph G is the m × m matrix defined as follows:

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j, \text{ that is, if there is an edge } (v_i, v_j) \\ 0 & \text{otherwise} \end{cases}$$

Such a matrix A, which contains entries of only 0 and 1, is called a *bit matrix* or a *Boolean matrix*.

The adjacency matrix A of the graph G does depend on the ordering of the nodes of G; that is, a different ordering of the nodes may result in a different adjacency matrix. However, the matrices resulting from two different orderings are closely related in that one can be obtained from the other by simply interchanging rows and columns. Unless otherwise stated, we will assume that the nodes of our graph G have a fixed ordering.

Representation of Graphs

Example 8.3

Consider the graph G in Fig. 8.3. Suppose the nodes are stored in memory in a linear array DATA as follows:

DATA: X, Y, Z, W

Then we assume that the ordering of the nodes in G is as follows: $v_1 = X$, $v_2 = Y$, $v_3 = Z$ and $v_4 = W$. The adjacency matrix A of G is as follows:

$$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Note that the number of 1's in A is equal to the number of edges in G.

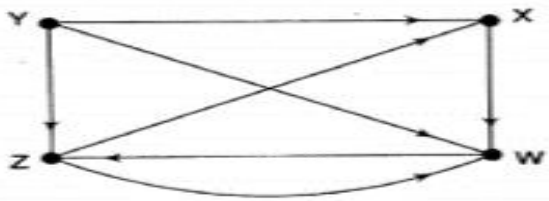


Fig. 8.3

- For a simple graph (that has no loops), the adjacency matrix has 0s on the diagonal.
- The adjacency matrix of an undirected graph is symmetric.
- The memory use of an adjacency matrix is $O(n^2)$, where n is the number of nodes in the graph.
- Number of 1s (or non-zero entries) in an adjacency matrix is equal to the number of edges in the graph.
- The adjacency matrix for a weighted graph contains the weights of the edges connecting the nodes.

Representation of Graphs

Proposition 8.2

Let A be the adjacency matrix of a graph G. Then $a_K(i, j)$, the ij entry in the matrix A^K, gives the number of paths of length K from v_i to v_j.

Consider again the graph G in Fig. 8.3, whose adjacency matrix A is given in Example 8.3. The powers A^2, A^3 and A^4 of the matrix A follow:

$$A^2 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 2 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \qquad A^3 = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 2 & 2 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix} \qquad A^4 = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 2 & 0 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

Accordingly, in particular, there is a path of length 2 from v_4 to v_1, there are two paths of length 3 from v_2 to v_3, and there are three paths of length 4 from v_2 to v_4. (Here, $v_1 = X$, $v_2 = Y$, $v_3 = Z$ and $v_4 = W$.)

Suppose we now define the matrix B_r as follows:

$$B_r = A + A^2 + A^3 + \cdots + A^r$$

Then the ij entry of the matrix B_r gives the number of paths of length r or less from node v_i to v_j.

Representation of Graphs

Path Matrix

Let G be a simple directed graph with m nodes, $v_1, v_2, \ldots, v_m$. The *path matrix* or *reachability matrix* of G is the m-square matrix $P = (p_{ij})$ defined as follows:

$$P_{ij} = \begin{cases} 1 & \text{if there is a path from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$$

Suppose there is a path from v_i to v_j. Then there must be a simple path from v_i to v_j when $v_i \neq v_j$, or there must be a cycle from v_i to v_j when $v_i = v_j$. Since G has only m nodes, such a simple path must have length $m - 1$ or less, or such a cycle must have length m or less. This means that there is a nonzero ij entry in the matrix B_m, defined at the end of the preceding subsection. Accordingly, we have the following relationship between the path matrix P and the adjacency matrix A.

Proposition 8.3

Let A be the adjacency matrix and let $P = (p_{ij})$ be the path matrix of a digraph G. Then $P_{ij} = 1$ if and only if there is a nonzero number in the ij entry of the matrix

$$B_m = A + A^2 + A^3 + \cdots + A^m$$

Consider the graph G with $m = 4$ nodes in Fig. 8.3. Adding the matrices A, A^2, A^3 and A^4, we obtain the following matrix B_4, and, replacing the nonzero entries in B_4 by 1, we obtain the path matrix P of the graph G:

$$B_4 = \begin{pmatrix} 1 & 0 & 2 & 3 \\ 5 & 0 & 6 & 8 \\ 3 & 0 & 3 & 5 \\ 2 & 0 & 3 & 3 \end{pmatrix} \quad \text{and} \quad P = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

Examining the matrix P, we see that the node v_2 is not reachable from any of the other nodes.

Recall that a directed graph G is said to be *strongly connected* if, for any pair of nodes u and v in G, there are both a path from u to v and a path from v to u. Accordingly, G is strongly connected if and only if the path matrix P of G has no zero entries. Thus the graph G in Fig. 8.3 is not strongly connected.

WARSHALL'S ALGORITHM

If a graph G is given as G=(V, E), where V is the set of vertices and E is the set of edges, the path matrix of G can be found as, $P = A + A^2 + A^3 + ... + A^n$. This is a lengthy process. Warshall has given a very efficient algorithm to calculate the path matrix. Warshall's algorithm defines matrices P0 , P1 , P2 , … , Pn as given in Fig.

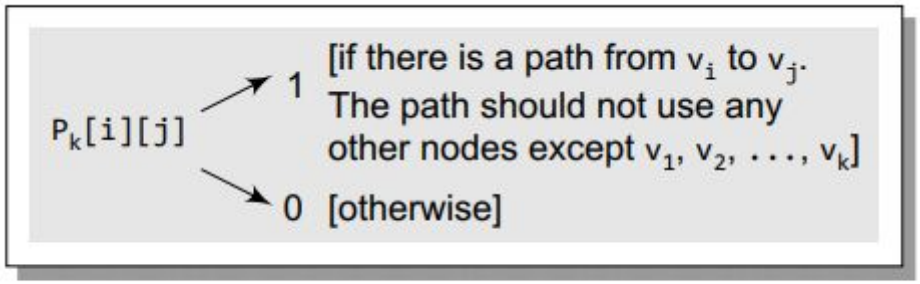


Figure 13.37 Path matrix entry

P_k [i][j] is equal to 1 only when either of the two following cases occur:

1. There is a path from v_i to v_j that does not use any other node except v_1 , v_2 , ..., v_{k-1}. Therefore, $P_{k-1}[i][j] = 1$.
2. There is a path from v_i to v_k and a path from v_k to v_j where all the nodes use v_1 , v_2 , ..., v_{k-1}.

Therefore, $P_{k-1}[i][k] = 1$ AND $P_{k-1}[k][j] = 1$

WARSHALL'S ALGORITHM

Hence, the path matrix Pn can be calculated with the formula given as: $P_k[i][j] = P_{k-1}[i][j] \vee (P_{k-1}[i][k] \wedge P_{k-1}[k][j])$
where V indicates logical OR operation and Λ indicates logical AND operation.

Algorithm 8.1: (Warshall's Algorithm) A directed graph G with M nodes is maintained in memory by its adjacency matrix A. This algorithm finds the (Boolean) path matrix P of the graph G.

1. Repeat for I, J = 1, 2,..., M: [Initializes P.]
   If A[I, J] = 0, then: Set P[I, J] := 0;
   Else: Set P[I, J] := 1.
   [End of loop.]
2. Repeat Steps 3 and 4 for K = 1, 2, ..., M: [Updates P.]
3. Repeat Step 4 for I = 1, 2, ..., M:
4. Repeat for J = 1, 2, ..., M:
   Set P[I, J] : = P[I, J] ∨ (P[I, K] ∧ P[K, J]).
   [End of loop.]
   [End of Step 3 loop.]
   [End of Step 2 loop.]
5. Exit.

WARSHALL'S ALGORITHM

8.8 Consider the graph G in Fig. 8.21 and its adjacency matrix A obtained in Problem 8.7. Find the path matrix P of G using Warshall's algorithm rather than the powers of A.

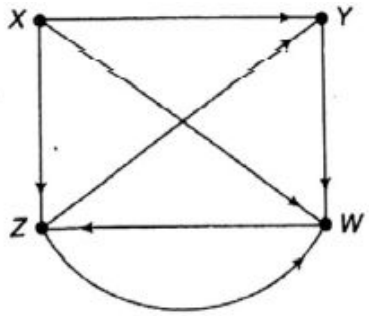


Fig. 8.21

Then:

$$P_1 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \qquad P_2 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$P_3 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix} \qquad P_4 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

Observe that $P_0 = P_1 = P_2 = A$. The changes in P_3 occur for the following reasons:

$P_3(4, 2) = 1$	because	$P_2(4, 3) = 1$	and	$P_2(3, 2) = 1$
$P_3(4, 4) = 1$	because	$P_2(4, 3) = 1$	and	$P_2(3, 4) = 1$

The changes in P_4 occur similarly. The last matrix, P_4, is the required path matrix P of the graph G.

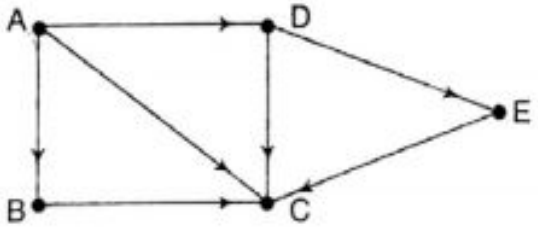
Lecture – 17

Graphs-2

Adjacency List

Let G be a directed graph with m nodes. The sequential representation of G in memory—i.e., the representation of G by its adjacency matrix A—has a number of major drawbacks. First of all, it may be difficult to insert and delete nodes in G. This is because the size of A may need to be changed and the nodes may need to be reordered, so there may be many, many changes in the matrix A. Furthermore, if the number of edges is $O(m)$ or $O(m \log_2 m)$, then the matrix A will be sparse (will contain many zeros); hence a great deal of space will be wasted. Accordingly, G is usually represented in memory by a linked *representation*, also called an *adjacency structure*, which is described in this section.

Consider the graph G in Fig. 8.7(a). The table in Fig. 8.7(b) shows each node in G followed by its *adjacency list*, which is its list of adjacent nodes, also called its *successors* or *neighbors*. Figure 8.8 shows a schematic diagram of a linked representation of G in memory. Specifically, the linked representation will contain two lists (or files), a node list NODE and an edge list EDGE, as follows.



(a) Graph G

Node	Adjacency List
A	B, C, D
B	C
C	
D	C, E
E	C

(b) Adjacency list of G

Fig. 8.7

Linked Representation of Graphs

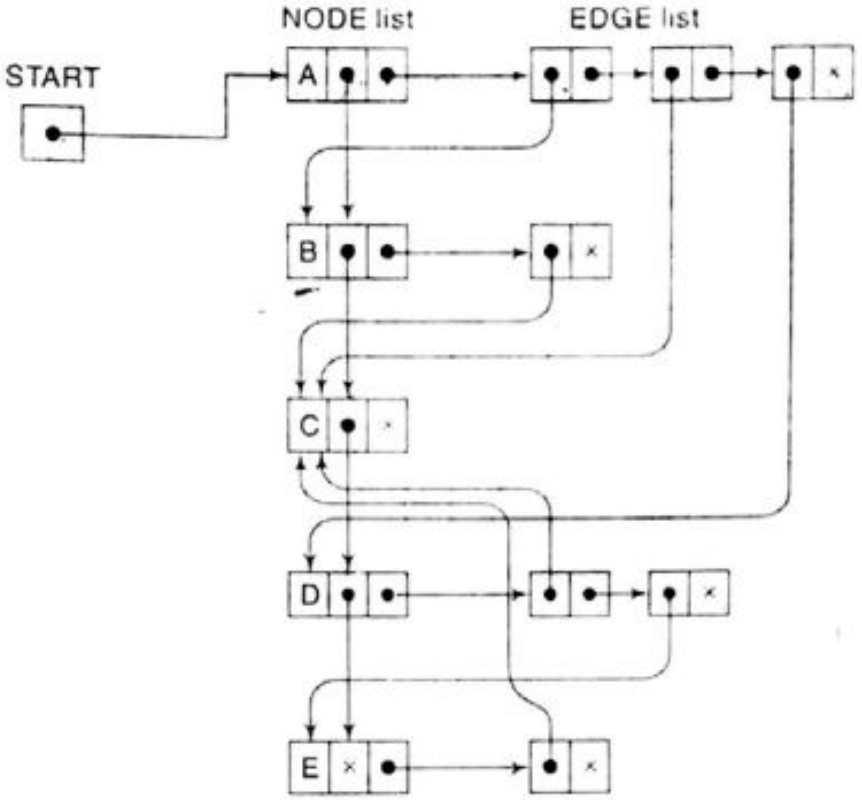


Fig. 8.8

(a) *Node list.* Each element in the list NODE will correspond to a node in G, and it will be a record of the form:

NODE	NEXT	ADJ	

Here NODE will be the name or key value of the node, NEXT will be a pointer to the next node in the list NODE and ADJ will be a pointer to the first element in the adjacency list of the node, which is maintained in the list EDGE. The shaded area indicates that there may be

Linked Representation of Graphs

(b) *Edge list.* Each element in the list EDGE will correspond to an edge of G and will be a record of the form:

DEST	LINK	

The field DEST will point to the location in the list NODE of the destination or terminal node of the edge. The field LINK will link together the edges with the same initial node, that is, the nodes in the same adjacency list. The shaded area indicates that there may be other information in the record corresponding to the edge, such as a field EDGE containing the labeled data of the edge when G is a labeled graph, a field WEIGHT containing the weight of the edge when G is a weighted graph, and so on. We also need a pointer variable AVAILE for the list of available space in the list EDGE.

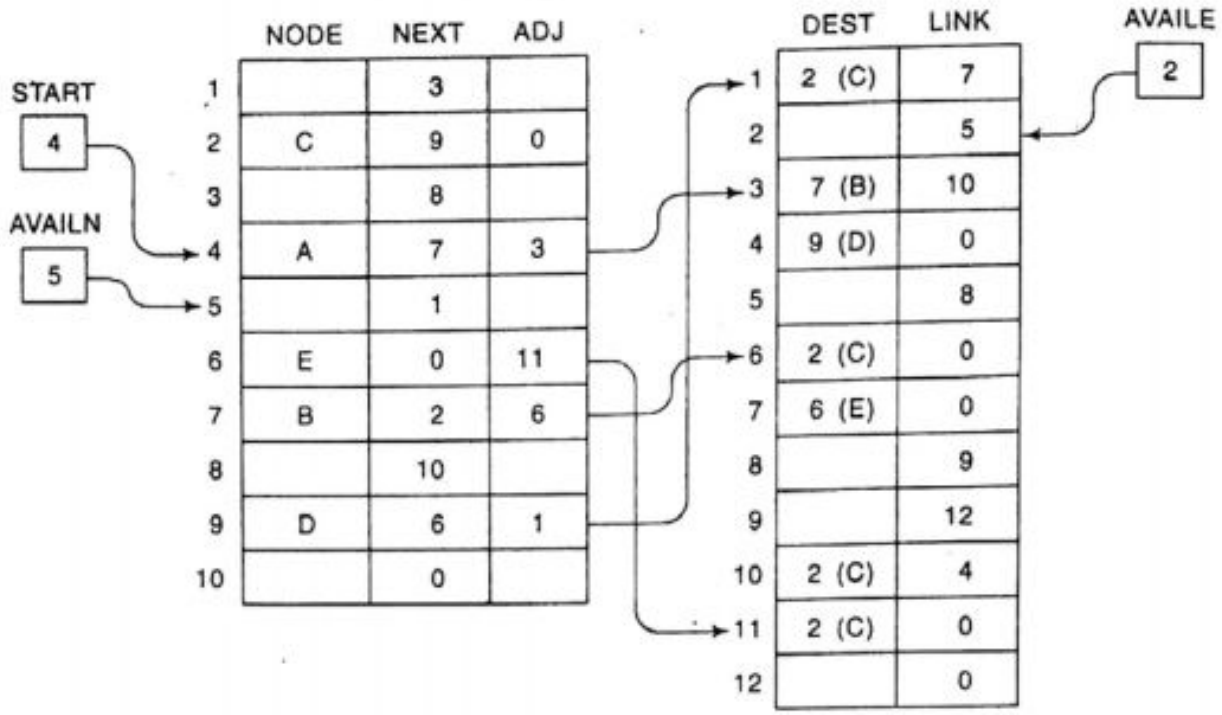


Fig. 8.9

Traversing A Graph

8.7 TRAVERSING A GRAPH

Many graph algorithms require one to systematically examine the nodes and edges of a graph G. There are two standard ways that this is done. One way is called a breadth-first search, and the other is called a depth-first search. The breadth-first search will use a queue as an auxiliary structure to hold nodes for future processing, and analogously, the depth-first search will use a stack.

During the execution of our algorithms, each node N of G will be in one of three states, called the *status* of N, as follows:

STATUS = 1: (Ready state.) The initial state of the node N.
STATUS = 2: (Waiting state.) The node N is on the queue or stack, waiting to be processed.
STATUS = 3: (Processed state.) The node N has been processed.

We now discuss the two searches separately.

Traversing A

Breadth-First Search

The general idea behind a breadth-first search beginning at a starting node A is as follows. First we examine the starting node A. Then we examine all the neighbors of A. Then we examine all the neighbors of the neighbors of A. And so on. Naturally, we need to keep track of the neighbors of a node, and we need to guarantee that no node is processed more than once. This is accomplished by using a queue to hold nodes that are waiting to be processed, and by using a field STATUS which tells us the current status of any node. The algorithm follows.

Algorithm A: This algorithm executes a breadth-first search on a graph G beginning at a starting node A.

1. Initialize all nodes to the ready state (STATUS = 1).
2. Put the starting node A in QUEUE and change its status to the waiting state (STATUS = 2).
3. Repeat Steps 4 and 5 until QUEUE is empty:
4. Remove the front node N of QUEUE. Process N and change the status of N to the processed state (STATUS = 3).
5. Add to the rear of QUEUE all the neighbors of N that are in the steady state (STATUS = 1), and change their status to the waiting state (STATUS = 2).

[End of Step 3 loop.]

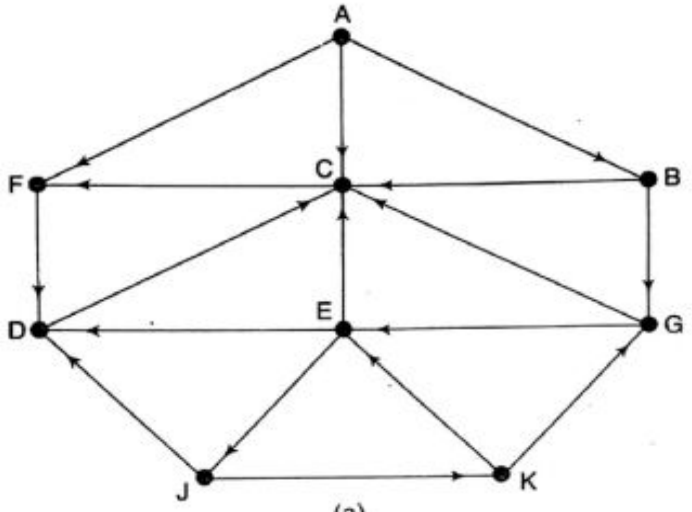
6. Exit.

The above algorithm will process only those nodes which are reachable from the starting node A. Suppose one wants to examine all the nodes in the graph G. Then the algorithm must be modified so that it begins again with another node (which we will call B) that is still in the ready state. This node B can be obtained by traversing the list of nodes.

Traversing A Graph

Example 8.7

Consider the graph G in Fig. 8.14(a). (The adjacency lists of the nodes appear in Fig. 8.14(*b*).) Suppose G represents the daily flights between cities of some airline, and suppose we want to fly from city A to city J with the minimum number of stops. In other words, we want the minimum path *P* from A to J (where each edge has length 1).



(a)

Adjacency lists	
A:	F, C, B
B:	G, C
C:	F
D:	C
E:	D, C, J
F:	D
G:	C, E
J:	D, K
K:	E, G

(b)

Fig. 8.14

(a) Initially, add A to QUEUE and add NULL to ORIG as follows:

FRONT = 1 QUEUE: A
REAR = 1 ORIG : Ø

(b) Remove the front element A from QUEUE by setting FRONT := FRONT + 1, and add to QUEUE the neighbors of A as follows:

FRONT = 2 QUEUE: A, F, C, B
REAR = 4 ORIG : Ø, A, A, A

Note that the origin A of each of the three edges is added to ORIG.

(c) Remove the front element F from QUEUE by setting FRONT := FRONT + 1, and add to QUEUE the neighbors of F as follows:

FRONT = 3 QUEUE: A, F, C, B, D
REAR = 5 ORIG : Ø, A, A, A, F

Traversing A Graph

(d) Remove the front element C from QUEUE, and add to QUEUE the neighbors of C (which are in the ready state) as follows:

```
FRONT = 4    QUEUE: A, F, C, B, D
REAR  = 5    ORIG  : Ø, A, A, A, F
```

Note that the neighbor F of C is not added to QUEUE, since F is not in the ready state (because F has already been added to QUEUE).

(e) Remove the front element B from QUEUE, and add to QUEUE the neighbors of B (the ones in the ready state) as follows:

```
FRONT = 5    QUEUE: A, F, C, B, D, G
REAR  = 6    ORIG  : Ø, A, A, A, F, B
```

Note that only G is added to QUEUE, since the other neighbor, C is not in the ready state.

(f) Remove the front element D from QUEUE, and add to QUEUE the neighbors of D (the ones in the ready state) as follows:

```
FRONT = 6    QUEUE: A, F, C, B, D, G
REAR  = 6    ORIG  : Ø, A, A, A, F, B
```

(g) Remove the front element G from QUEUE and add to QUEUE the neighbors of G (the ones in the ready state) as follows:

```
FRONT = 7    QUEUE: A, F, C, B, D, G, E
REAR  = 7    ORIG  : Ø, A, A, A, F, B, G
```

(h) Remove the front element E from QUEUE and add to QUEUE the neighbors of E (the ones in the ready state) as follows:

```
FRONT = 8    QUEUE: A, F, C, B, D, G, E, J
REAR  = 8    ORIG  : Ø, A, A, A, F, B, G, E
```

We stop as soon as J is added to QUEUE, since J is our final destination. We now backtrack from J, using the array ORIG to find the path *P*. Thus

$$J \leftarrow E \leftarrow G \leftarrow B \leftarrow A$$

is the required path *P*.

Traversing A Graph

Depth-First Search

The general idea behind a depth-first search beginning at a starting node A is as follows. First we examine the starting node A. Then we examine each node N along a path P which begins at A; that is, we process a neighbor of A, then a neighbor of a neighbor of A, and so on. After coming to a "dead end," that is, to the end of the path P, we backtrack on P until we can continue along another, path P'. And so on. (This algorithm is similar to the inorder traversal of a binary tree, and the algorithm is also similar to the way one might travel through a maze.) The algorithm is very similar to the breadth-first search except now we use a stack instead of the queue. Again, a field STATUS is used to tell us the current status of a node. The algorithm follows.

Algorithm B: This algorithm executes a depth-first search on a graph G beginning at a starting node A.

1. Initialize all nodes to the ready state (STATUS = 1).
2. Push the starting node A onto STACK and change its status to the waiting state (STATUS = 2).
3. Repeat Steps 4 and 5 until STACK is empty.
4. Pop the top node N of STACK. Process N and change its status to the processed state (STATUS = 3).
5. Push onto STACK all the neighbors of N that are still in the ready state (STATUS = 1), and change their status to the waiting state (STATUS = 2).

   [End of Step 3 loop.]
6. Exit.

Again, the above algorithm will process only those nodes which are reachable from the starting node A. Suppose one wants to examine all the nodes in G. Then the algorithm must be modified so that it begins again with another node which we will call B—that is still in the ready state. This node B can be obtained by traversing the list of nodes.

Traversing A Graph

Example 8.8

Consider the graph *G* in Fig. 8.14(a). Suppose we want to find and print all the nodes reachable from the node J (including J itself). One way to do this is to use a depth-first search of *G* starting at the node J. The steps of our search follow.

(a) Initially, push J onto the stack as follows:

STACK: J

(b) Pop and print the top element J, and then push onto the stack all the neighbors of J (those that are in the ready state) as follows:

Print J STACK: D, K

(c) Pop and print the top element K, and then push onto the stack all the neighbors of K (those that are in the ready state) as follows:

Print K STACK: D, E, G

(d) Pop and print the top element G, and then push onto the stack all the neighbors of G (those in the ready state) as follows:

Print G STACK: D, E, C

Note that only C is pushed onto the stack, since the other neighbor, E, is not in the ready state (because E has already been pushed onto the stack).

(e) Pop and print the top element C, and then push onto the stack all the neighbors of C (those in the ready state) as follows:

Print C STACK: D, E, F

Traversing A Graph

(f) Pop and print the top element F, and then push onto the stack all the neighbors of F (those in the ready state) as follows:

Print F STACK: D, E

Note that the only neighbor D of F is not pushed onto the stack, since D is not in the ready state (because D has already been pushed onto the stack).

(g) Pop and print the top element E, and push onto the stack all the neighbors of E (those in the ready state) as follows:

Print E STACK: D

(Note that none of the three neighbors of E is in the ready state.)

(h) Pop and print the top element D, and push onto the stack all the neighbors of D (those in the ready state) as follows:

Print D STACK:

The stack is now empty, so the depth-first search of *G* starting at J is now complete. Accordingly, the nodes which were printed,

J, K, G, C, F, E, D

are precisely the nodes which are reachable from J.

Differences between BFS and DFS

Basis for comparison	BFS	DFS
Definition	BFS stands for Breadth First Search	DFS stands for Depth First Search.
Basic	Vertex-based algorithm	Edge-based algorithm
Data structure used to store the nodes	Queue	Stack
Source	BFS is better when target is closer to Source.	DFS is better when target is far from source.
Suitability for decision tree	BFS considers all neighbors first and therefore not suitable for decision making trees used in games or puzzles.	DFS is more suitable for game or puzzle problems. We make a decision, then explore all paths through this decision. And if this decision leads to win situation, we stop.
Memory consumption	More memory BFS uses a larger amount of memory because it expands all children of a vertex and keeps them in memory. It stores the pointers to a level's child nodes while searching each level to remember where it should go when it reaches a leaf node.	Less memory DFS has much lower memory requirements than BFS because it iteratively expands only one child until it cannot proceed anymore and backtracks thereafter. It has to remember a single path with unexplored nodes.
Structure of the constructed tree	Wide and short	Narrow and long

Differences between BFS and DFS

Basis for comparison	BFS	DFS
Traversing fashion	Oldest unvisited vertices are explored at first.	Vertices along the edge are explored in the beginning.
Optimality	Optimal for finding the shortest distance, not in cost.	Not optimal
Application	Examines bipartite graph, connected component and shortest path present in a graph.	Examines two-edge connected graph, strongly connected graph, acyclic graph and topological order.
Time Complexity	The Time complexity of BFS is O(V + E) when Adjacency List is used and O(V^2) when Adjacency Matrix is used, where V stands for vertices and E stands for edges.	The Time complexity of DFS is also O(V + E) when Adjacency List is used and O(V^2) when Adjacency Matrix is used, where V stands for vertices and E stands for edges.